

Power Grid Load Forecasting using Machine Learning Approaches: Bachelor's Project

Supervised by Chanel Guillaume, Youssef Saïd, Clément Targe

Reason

By leveraging historical energy consumption data alongside weather and seasonal variables, this project aims to create accurate medium-term energy consumption forecasts and measure their effectiveness. This project adopts time series analysis techniques, like ARIMA and SARIMA models, for medium-term forecasting.

Medium-term forecasting plays a crucial role in balancing supply and demand, scheduling energy imports or hydropower reserves, and planning grid maintenance. Time series models provide interpretable, data-driven forecasts that align well with Switzerland's structured energy market and regulatory planning needs.

Objective

The primary objectives are:

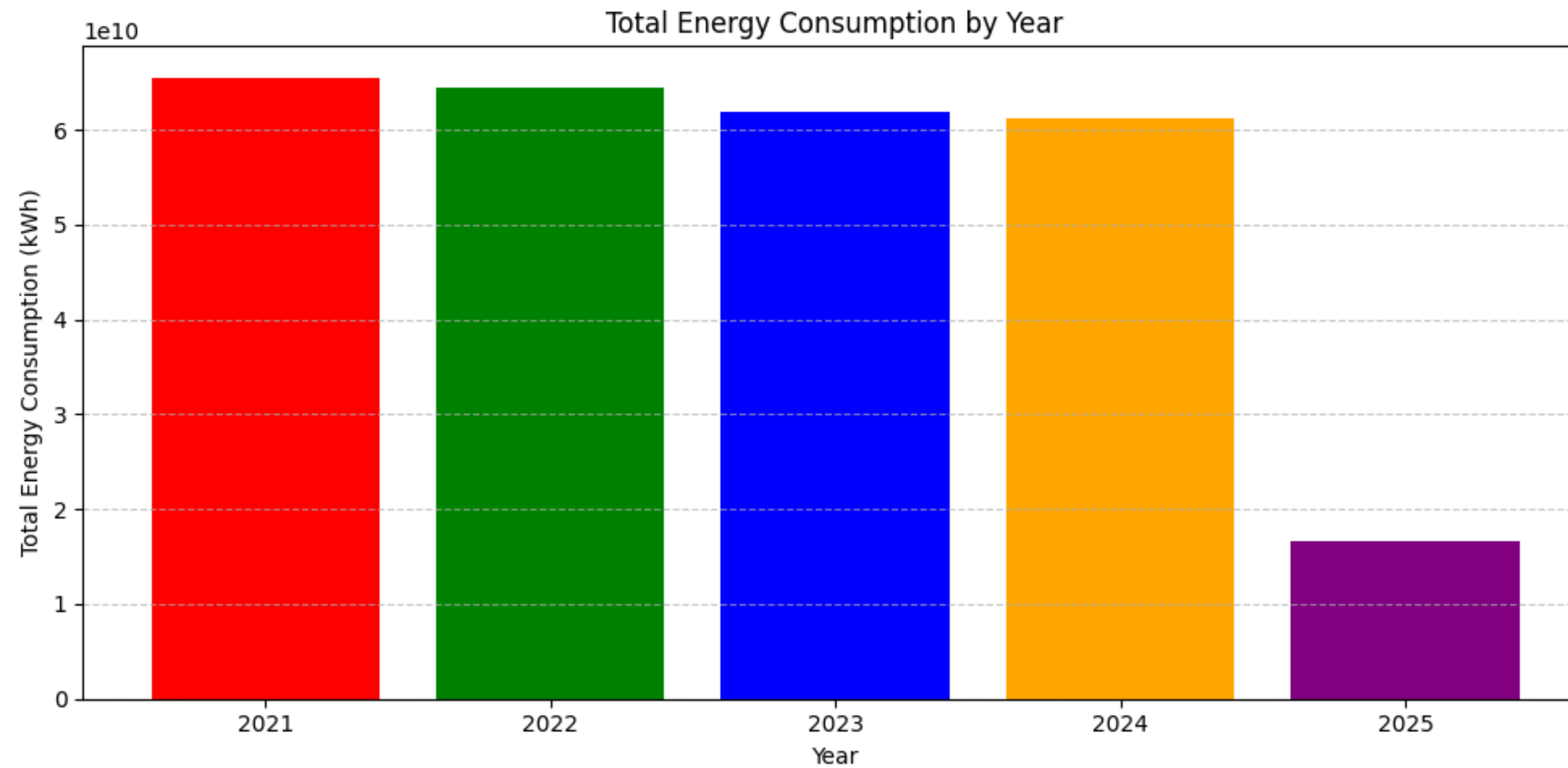
1. Visualize the Data
2. Develop a Medium Term Forecasting Model (Predicting Total Amount of Energy Consumed per day/week)
3. Setting a Baseline Model and model evaluation Metric
4. Evaluate the results
5. Discussing challenges and conclusions

Outline

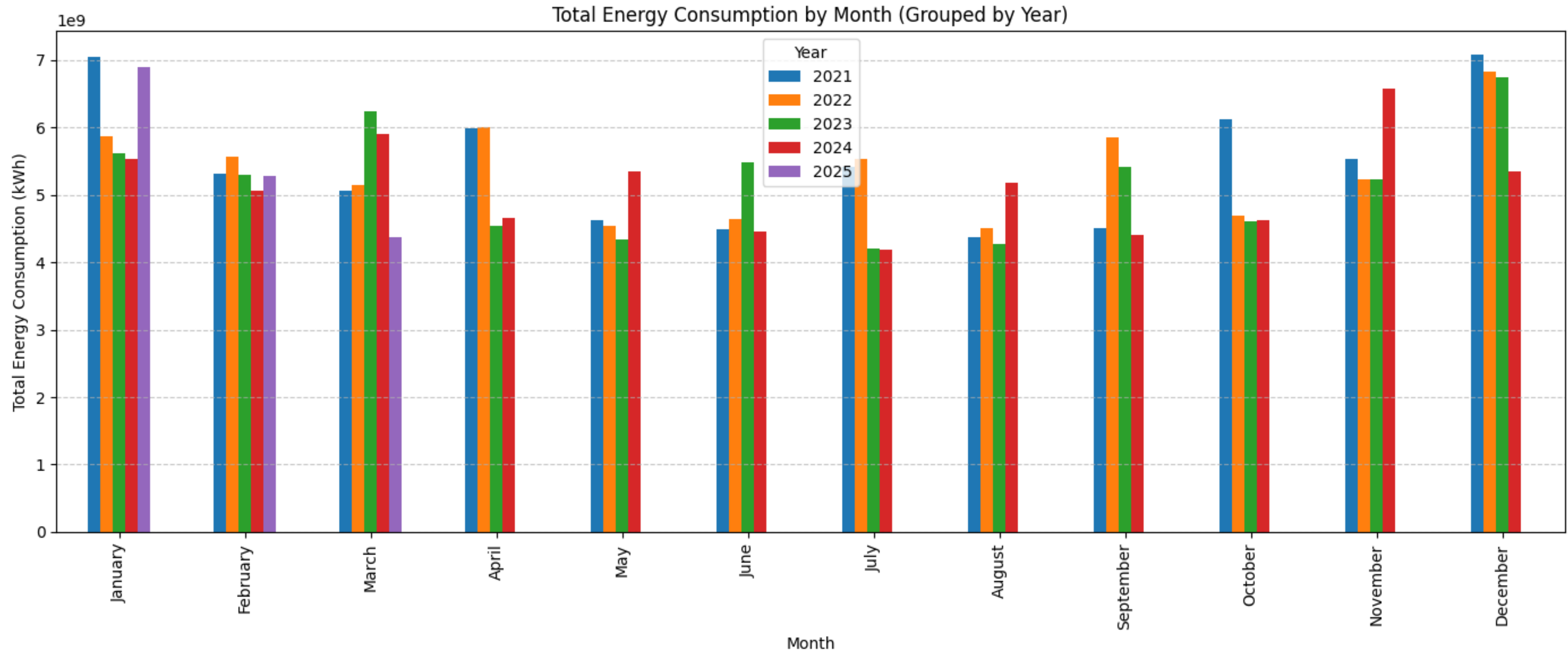
- I. Visualisation
- II. Data Cleaning
- III. Time Series Analysis
- IV. Models
- V. Evaluation
- VI. Challenges & Discussion

I. Visualisation

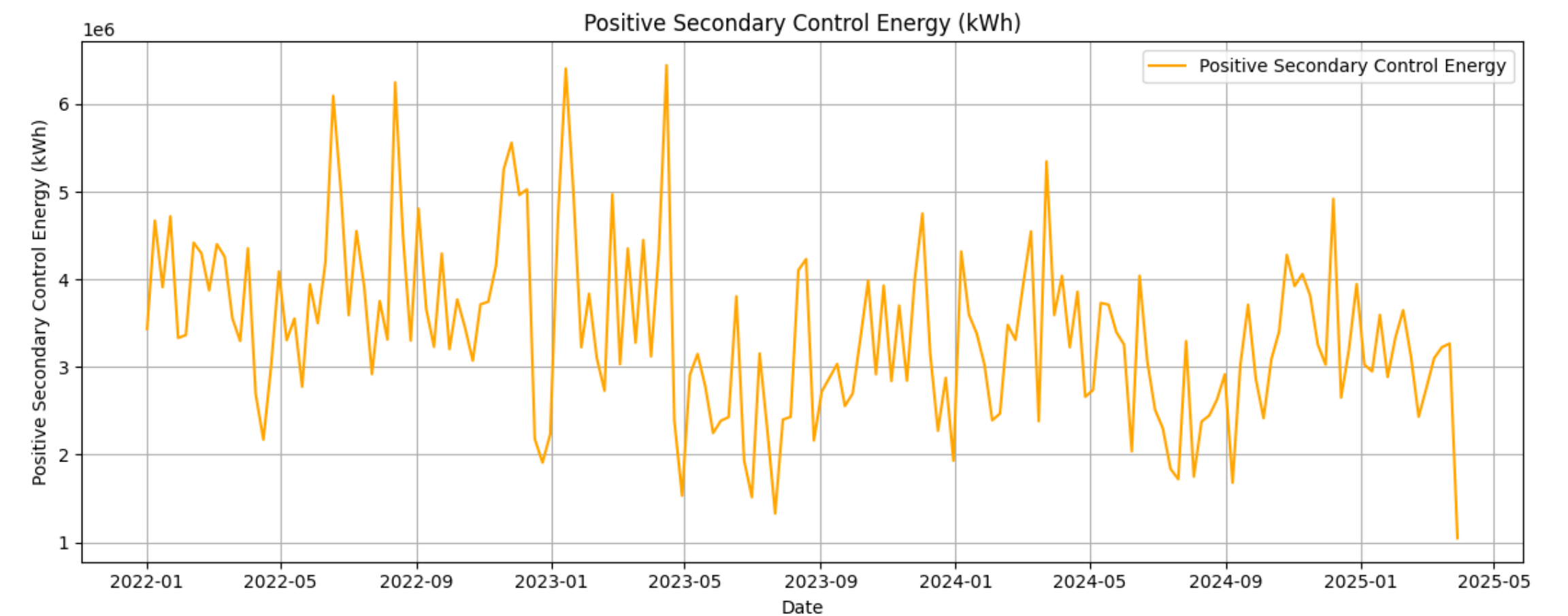
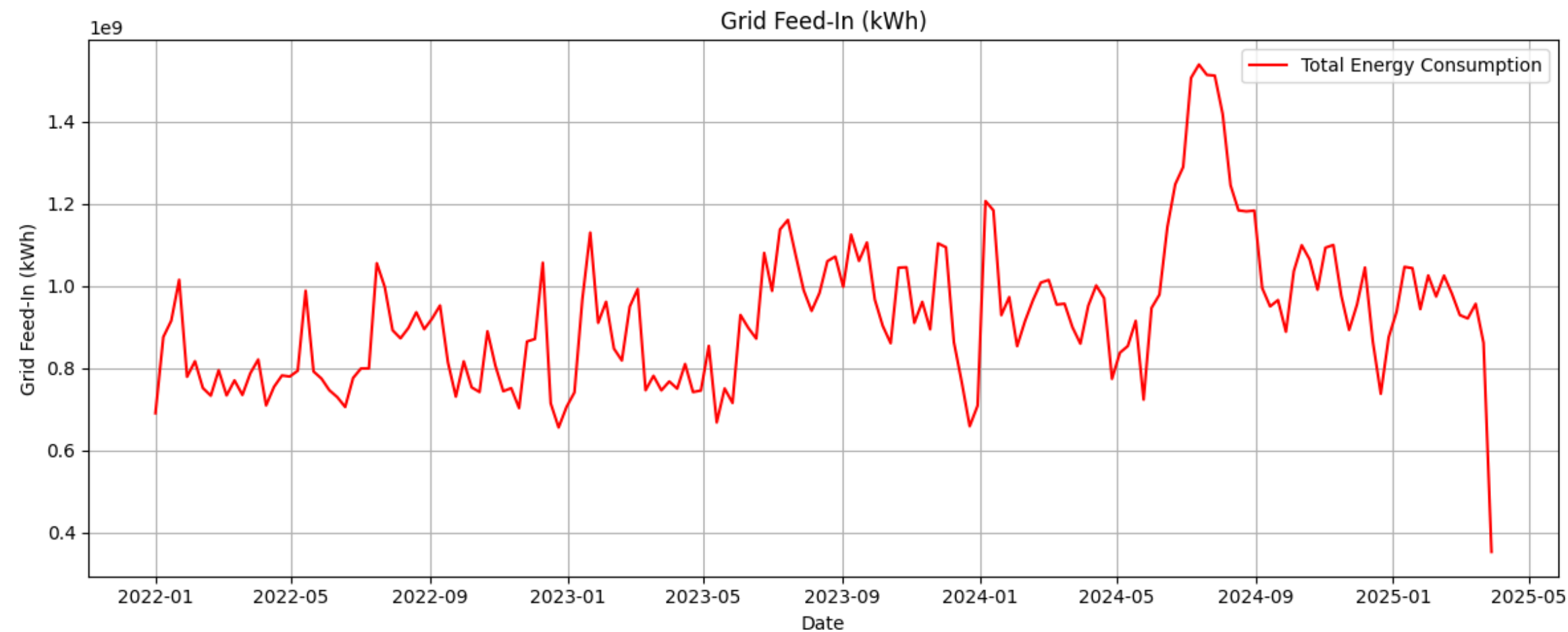
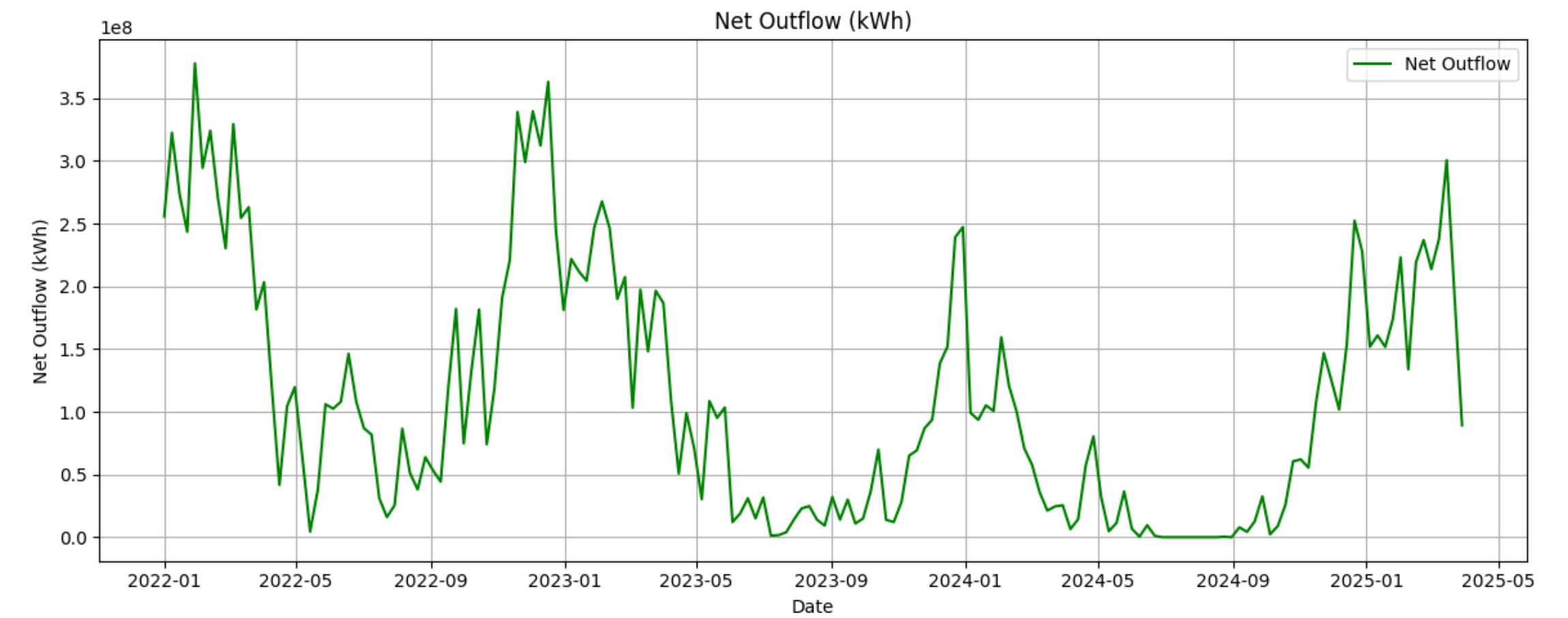
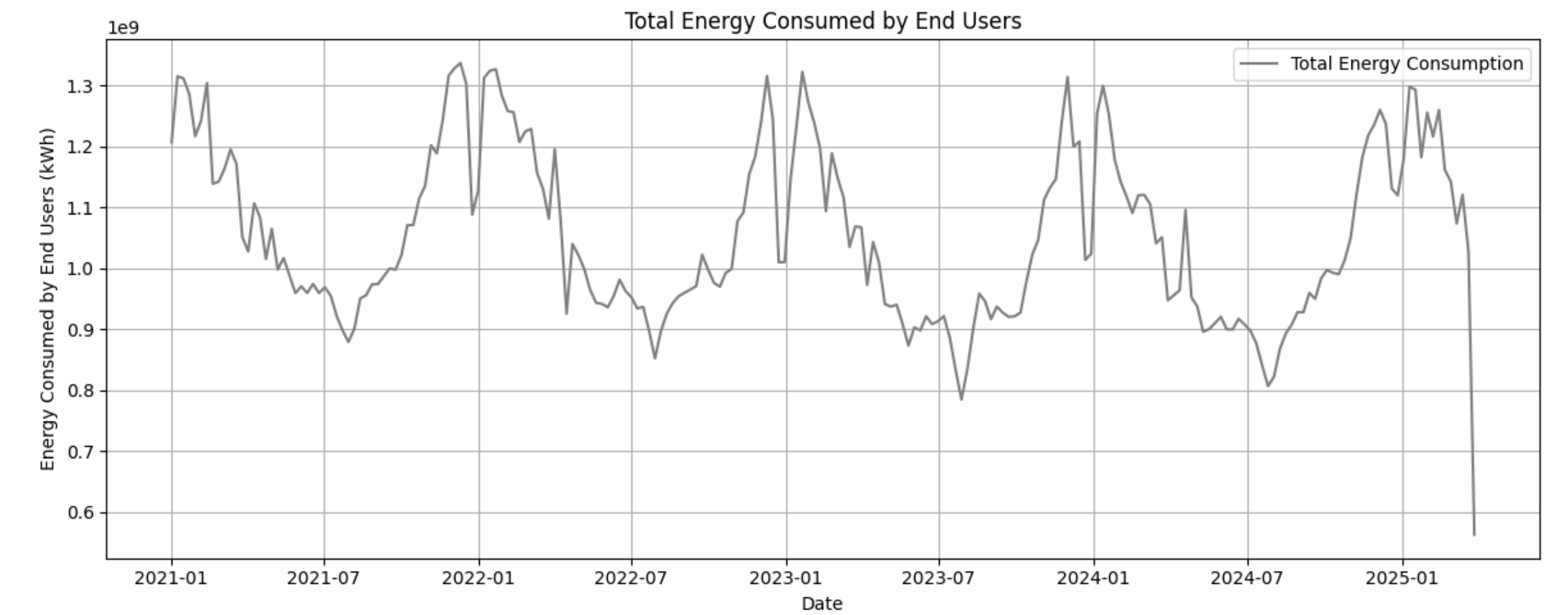
I. Visualisation



I. Visualisation



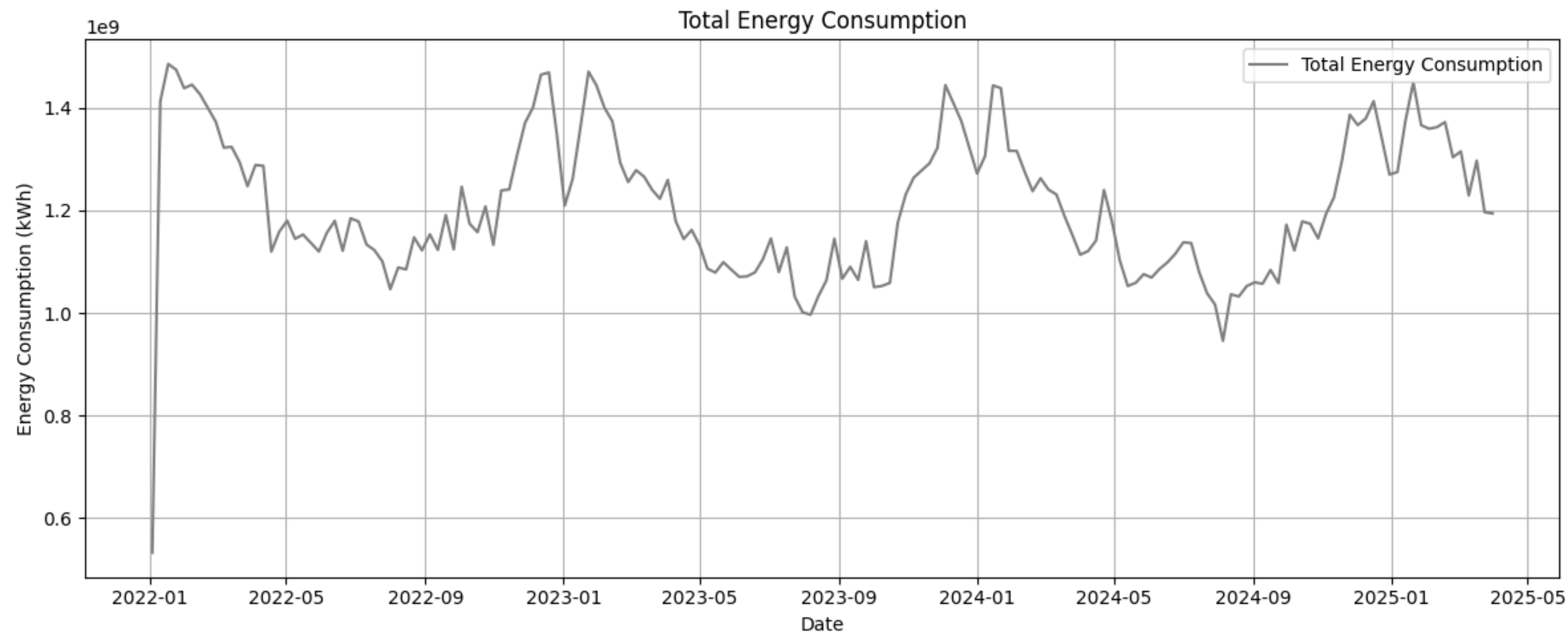
I. Visualisation of related variables



Variable to predict

Weekly sum

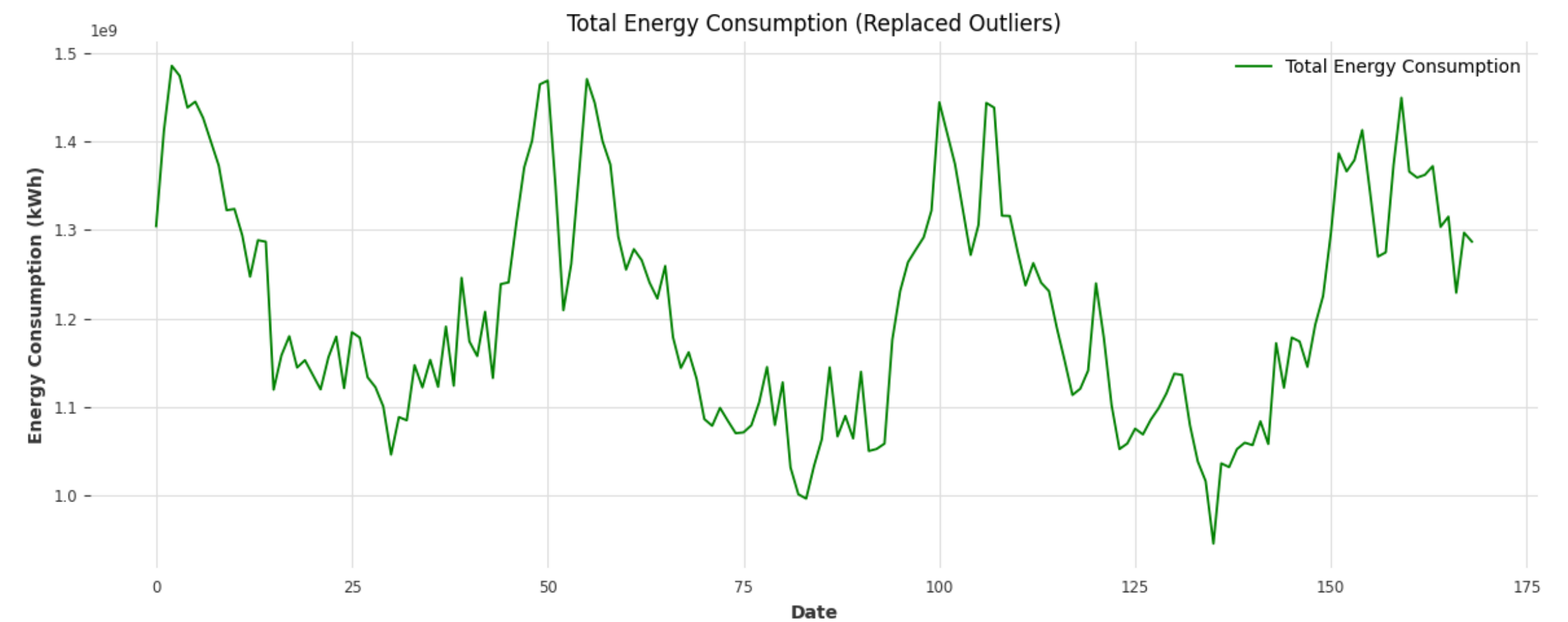
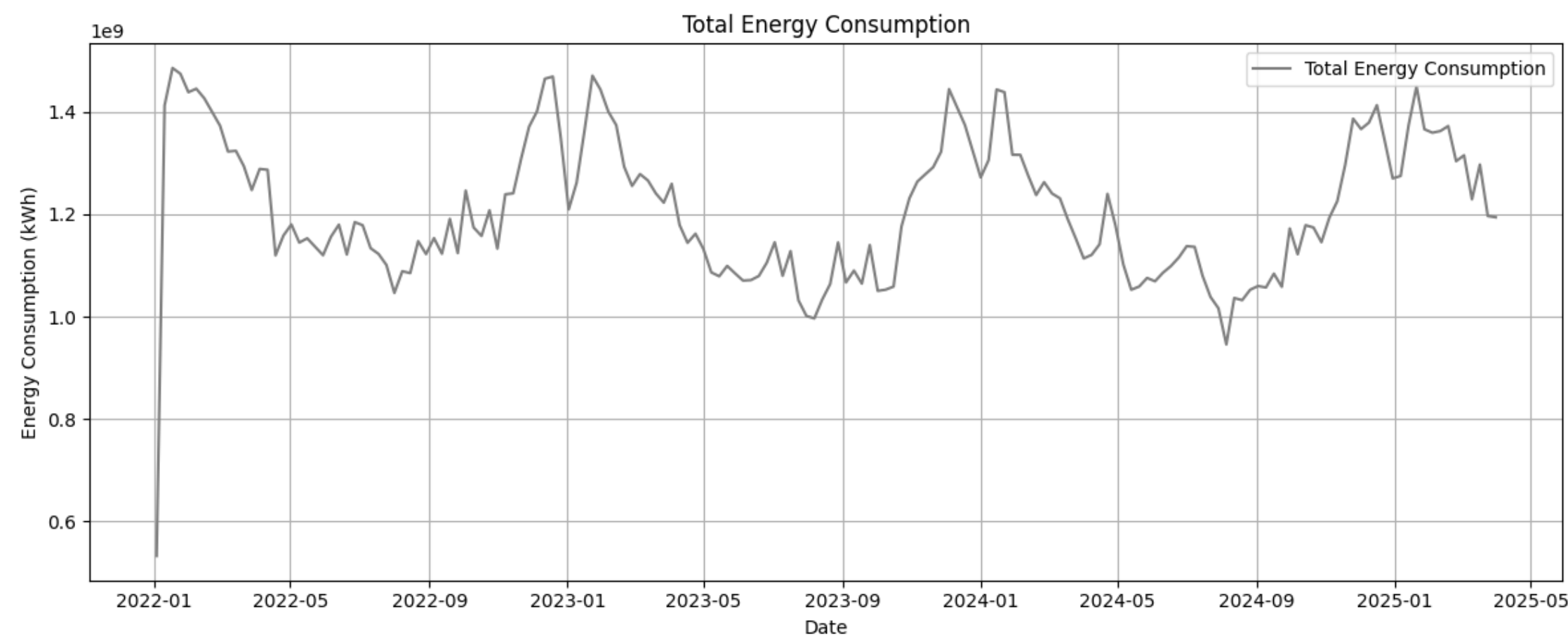
- The weekly predicted sum will be from Monday-Sunday, as is per EU energy regulations



Variable to predict

underreported values (self-caused)

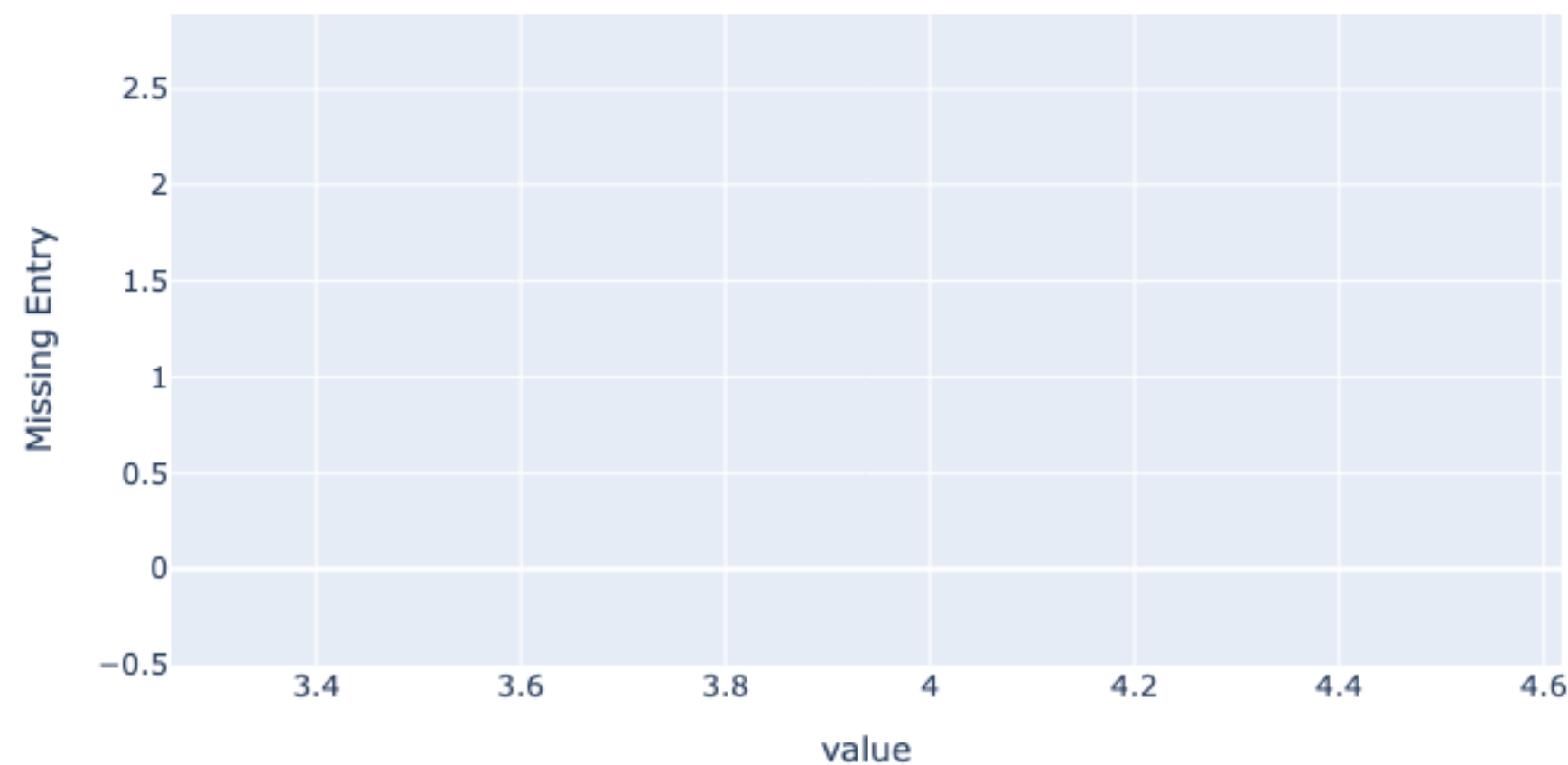
- After summing weekly from Monday, I removed the underreported values (first and last not fully summed values) and replaced them with the rolling mean in that period ($w=14$)



II. Data Cleaning

II. Data Cleaning

Scatterplot of the empty loads (Actual)



```
Check min, max, std for anomalies

print(df.describe())
✓ 0.2s

Total Energy Consumed by End Users (kWh) \
count      1.138520e+05
mean       1.557924e+06
std        2.965795e+05
min        8.924723e+05
25%        1.326569e+06
50%        1.545676e+06
75%        1.751344e+06
max        2.399117e+06
```

```
Is each index unique?

df.index.is_unique
[2536] ✓ 0.0s
... True
```

I cleaned the data by converting all values to numeric and checking for missing or invalid values. None were found.

I verified that each timestamp was unique.

Summary statistics looked normal, with no extreme outliers.

III. Time Series Analysis

- **III. Time Series Analysis**
 - I. ACF: which lags are most important, does my data have a moving average?
 - II. KPSS Test: is my data stationnary?
 - III. Periodogram: Predicting seasonality, and is my data white noise?
 - IV. After modeling, residual analysis, are they (white noise/normal)?

Autocovariance function

- Measures the linear dependence of a time series with a lagged version of itself
- Quantifies how much the values of a time series at different points in time are related to each other, specifically in terms of their covariance

The covariance function for equally spaced data $y_1, \dots, y_n$ is defined as:

$$c_h = \frac{1}{n-h-1} \sum_{i=1}^{n-h} (y_i - \bar{y})(y_{i+h} - \bar{y}), \quad h = 0, 1, \dots, n-2,$$

where $\bar{y}$ is the sample mean. The correlogram (ACF) is a graph of $\hat{\rho}_h = \frac{c_h}{c_0}$ against lag h [4].

Interpretation

Source

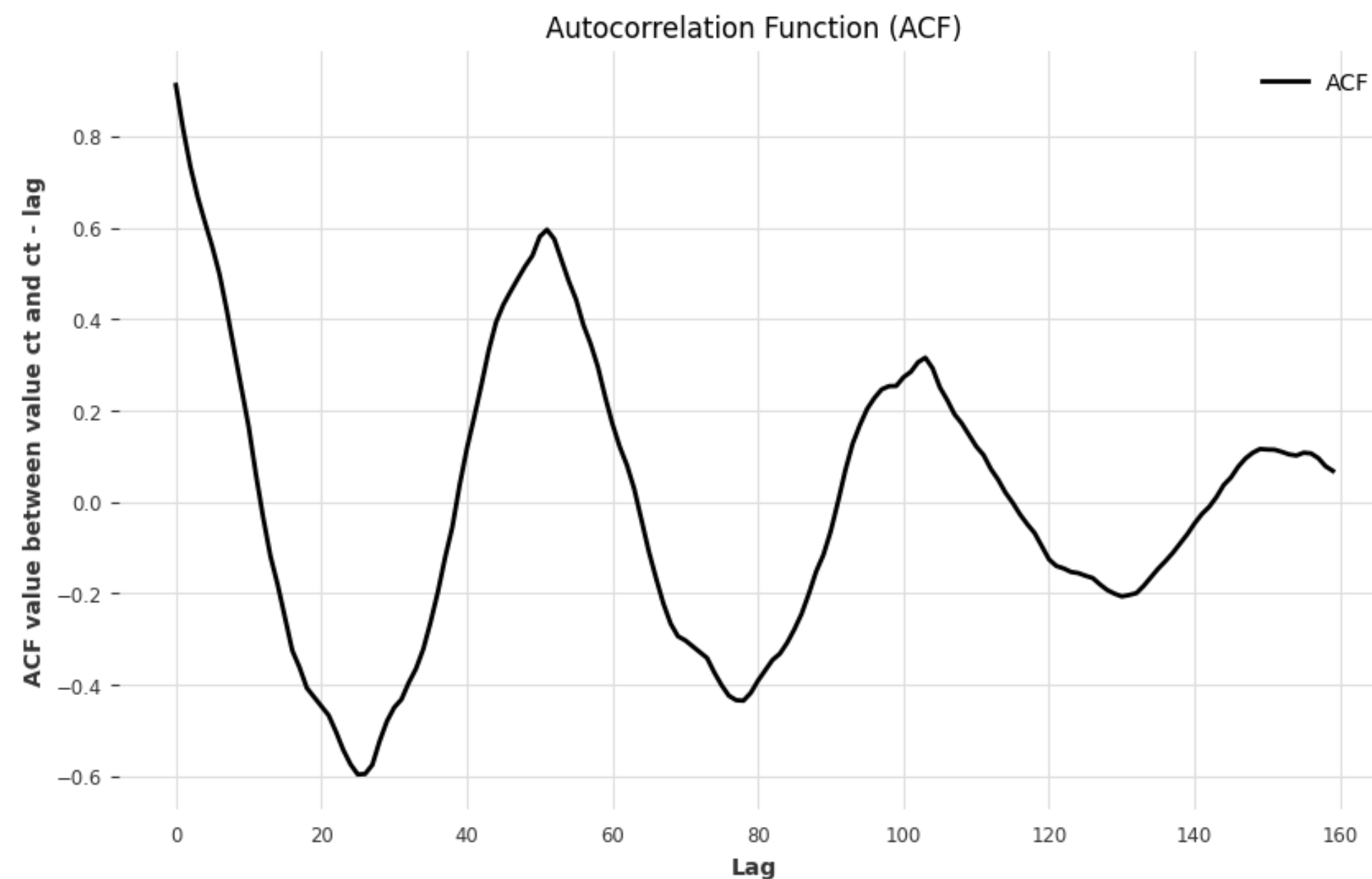
According to the Time Series Analysis book, To summarise: for causal and invertible ARMA models the ACF and PACF have the following properties:

	AR(p)	MA(q)	ARMA(p,q)
ACF	Tails off	Cuts off after lag q	Tails off
PACF	Cuts off after lag p	Tails off	Tails off

This gives an approach to identifying AR and MA models based on the ACF and PACF, and suggests how to choose p or q . [4]

Autocorrelation function of original data

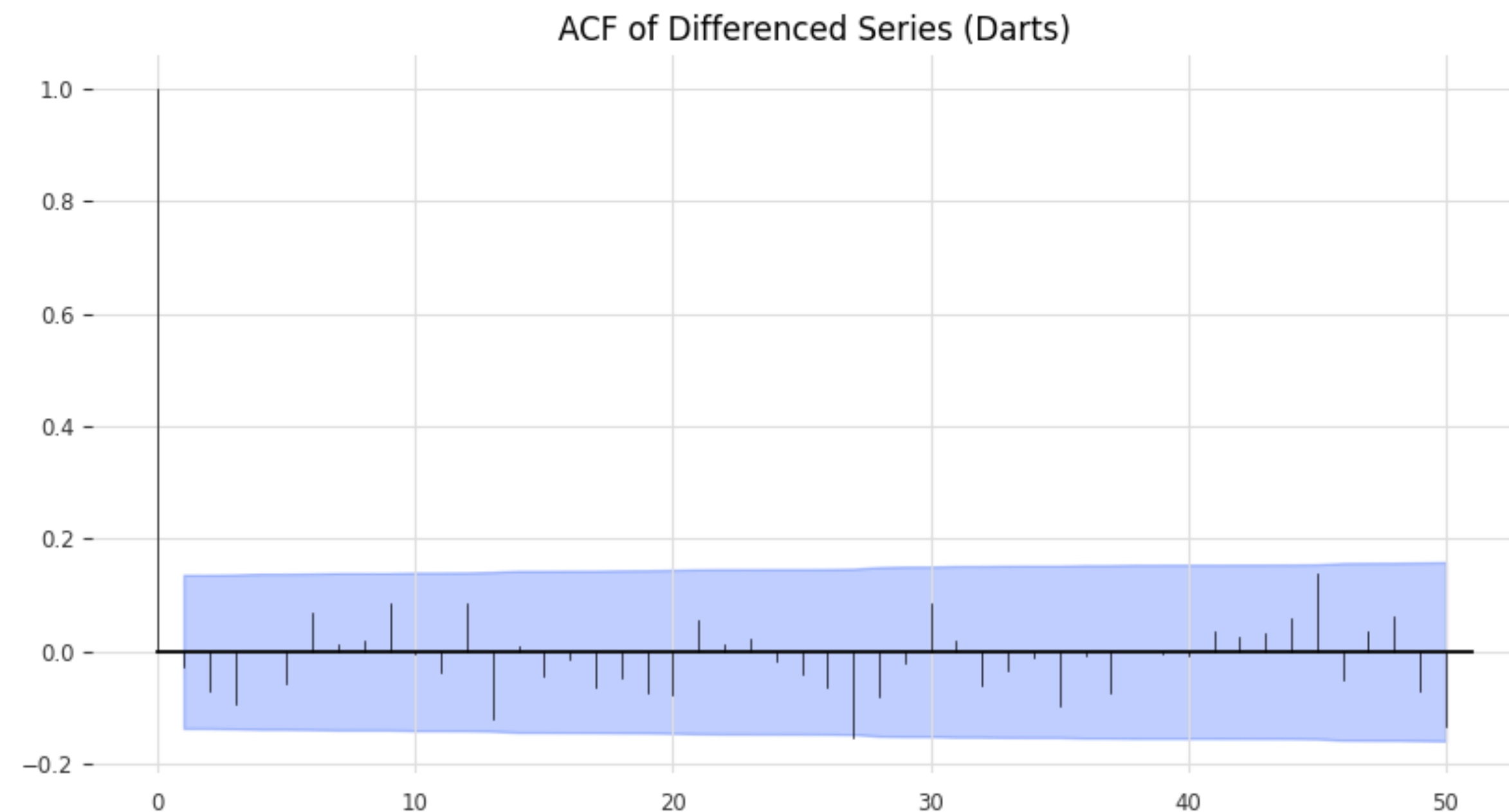
Plotted lags for 150-200



**ACF tails off, no sharp drop, is cyclical (=> potential seasonality)
High at lag 1, then rapidly decays, likely suggesting an AR(1) model
peaking every time at a 52-week cycle**

Testing for MA order

ACF of differenced data



The ACF values of the deseasoned ts show no significant cutout after a certain lag, this does not suggest a ARMA model

$$q = 0$$

Testing for stationarity

A stationary time series is a time series whose statistical properties like mean, variance, and autocorrelation are all constant over time

KPSS Test: The KPSS test is used to test the null hypothesis that a time series is stationary. It does this by estimating the test statistic:

$$C(l) = \frac{1}{\sigma^2(l)} \sum_{t=1}^n S_t^2, \quad \text{where } S_t = \sum_{j=1}^t e_j$$

Here, $e_1, \dots, e_n$ are the residuals from regressing Y_t on a constant or a linear trend (depending on whether testing for level or trend stationarity), and $\sigma^2(l)$ is a long-run variance estimate using a truncation lag l . [4] The test is interpreted as follows:

Based upon the significance level of 0.05 and the p-value of ADF test, the null hypothesis can not be rejected. Hence, the series is non-stationary.

It kpss result is low ==> stationary

Interpreting KPSS

KPSS test is now applied on the data.

[8]:

```
kpss_test(sunspots["SUNACTIVITY"])
```

Results of KPSS Test:

Test Statistic	0.669866
p-value	0.016285
Lags Used	7.000000
Critical Value (10%)	0.347000
Critical Value (5%)	0.463000
Critical Value (2.5%)	0.574000
Critical Value (1%)	0.739000
dtype:	float64

Based upon the significance level of 0.05 and the p-value of KPSS test, there is evidence for rejecting the null hypothesis in favor of the alternative. Hence, the series is non-stationary as per the KPSS test.

If kpss result is low ==> stationary (source: statsmodel)

Testing for stationarity

A stationary time series is a time series whose statistical properties like mean, variance, and autocorrelation are all constant over time

```
alpha=0.1
[2596] ✓ 0.0s

is_stationary = stationarity_test_kpss(initial_series)
stat, p_value, lags, crit_vals = stationarity_test_kpss(initial_series)
print(f"KPSS statistic: {stat}")
print(f"p-value: {p_value}")
is_stationary = p_value <= alpha
print(f"Is stationary: {is_stationary}")

[2597] ✓ 0.0s

... KPSS statistic: 0.11310880435373408
p-value: 0.1
Is stationary: True
```

```
from darts.utils.statistics import stationarity_tests

# Run both ADF and KPSS tests
is_stationary = stationarity_tests(
    initial_series,
    p_value_threshold_kpss=0.05
)

# Print interpretation
print("Series is stationary" if is_stationary else "Series is not stationary")

✓ 0.0s

Series is stationary
```

If kpss result is low ==> stationary
interpretation of p-value and kpss statistic, use kpss function instead
No need for ARIMA then (instead of ARMA), d=0

Linear Periodogram

Definition

- (a) If $y_1, \dots, y_n$ is an equally-spaced time series, its periodogram ordinate for ω is defined as

$$I(\omega) = |d(\omega_j)|^2$$

this means that:

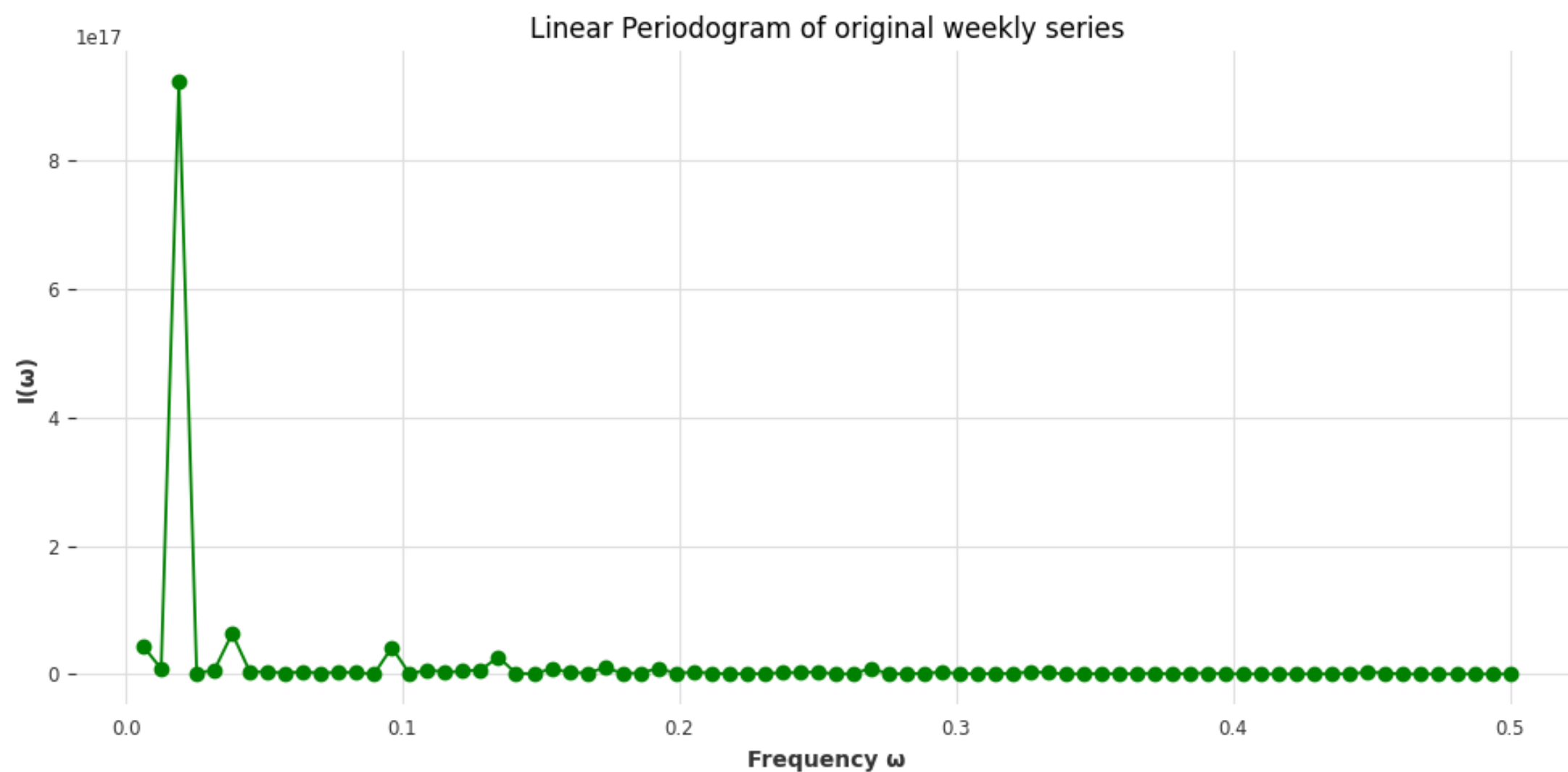
$$I(\omega) = \frac{1}{n} \left[\left(\sum_{t=1}^n y_t \cos(2\pi\omega t) \right)^2 + \left(\sum_{t=1}^n y_t \sin(2\pi\omega t) \right)^2 \right], \quad 0 < \omega \leq \frac{1}{2}$$

where $\omega_j = \frac{2\pi j}{n}$ and $j = 1, 2, \dots, n/2$

Linear Periodogram

Results

- The spectral decomposition shows that the highest frequency is of 52 weeks, this clearly demonstrates a yearly seasonality.
- The other frequencies are much weaker



```
significant_frequencies = [(w, I) for w, I in zip(freqs, I_vals) if I > threshold]
for freq, power in significant_frequencies:
    print(f"Frequency: {freq:.5f}, Power: {power:.2e}, Period ≈ {1/freq:.2f} weeks")
```

✓ 0.0s

```
Frequency: 0.00641, Power: 4.29e+16, Period ≈ 156.00 weeks
Frequency: 0.01923, Power: 9.24e+17, Period ≈ 52.00 weeks
Frequency: 0.03846, Power: 6.33e+16, Period ≈ 26.00 weeks
Frequency: 0.09615, Power: 4.15e+16, Period ≈ 10.40 weeks
Frequency: 0.13462, Power: 2.65e+16, Period ≈ 7.43 weeks
```

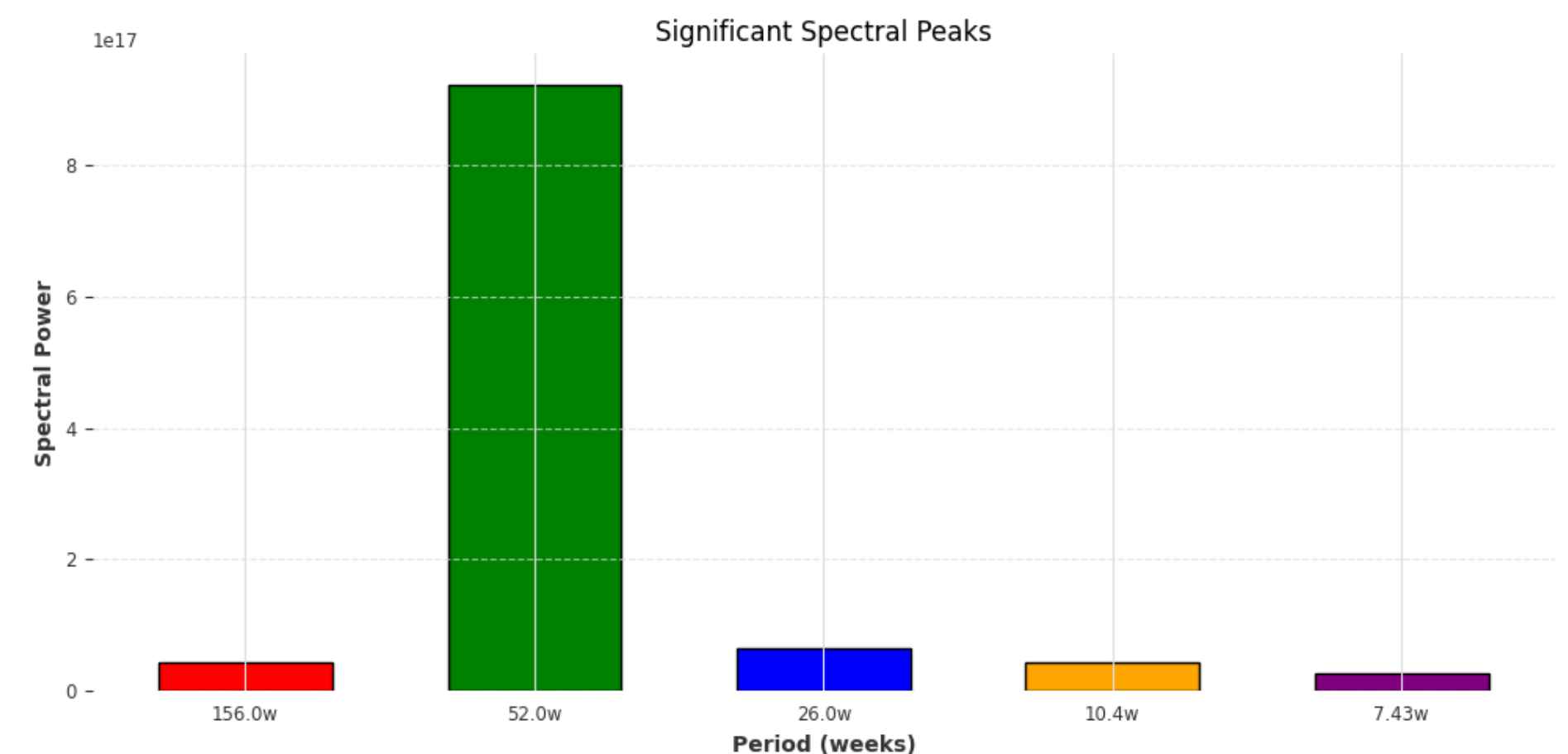

Spectral decomposition

Results

- The spectral decomposition shows that the highest frequency is of 52 weeks, this clearly demonstrates a yearly seasonality.
- The other frequencies are much weaker

```
significant_frequencies = [(w, I) for w, I in zip(freqs, I_vals) if I > threshold]
for freq, power in significant_frequencies:
    print(f"Frequency: {freq:.5f}, Power: {power:.2e}, Period ≈ {1/freq:.2f} weeks")
✓ 0.0s
```

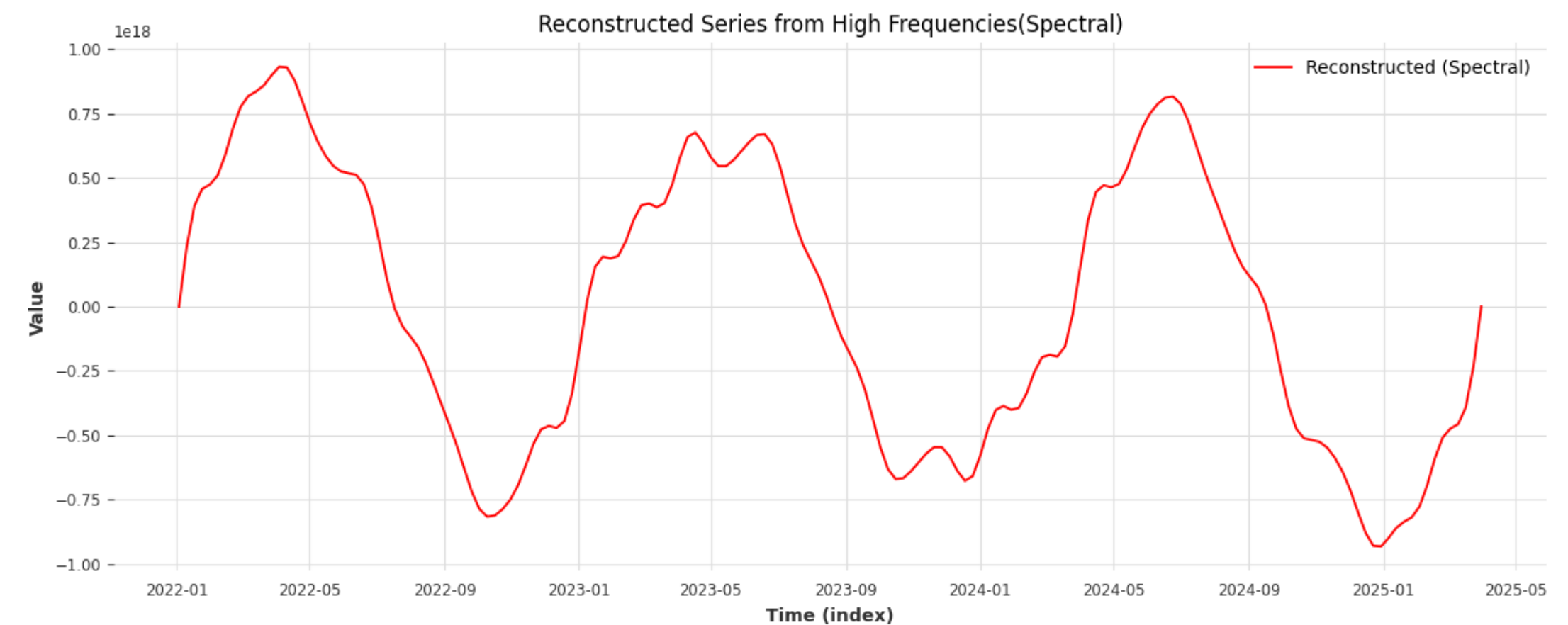
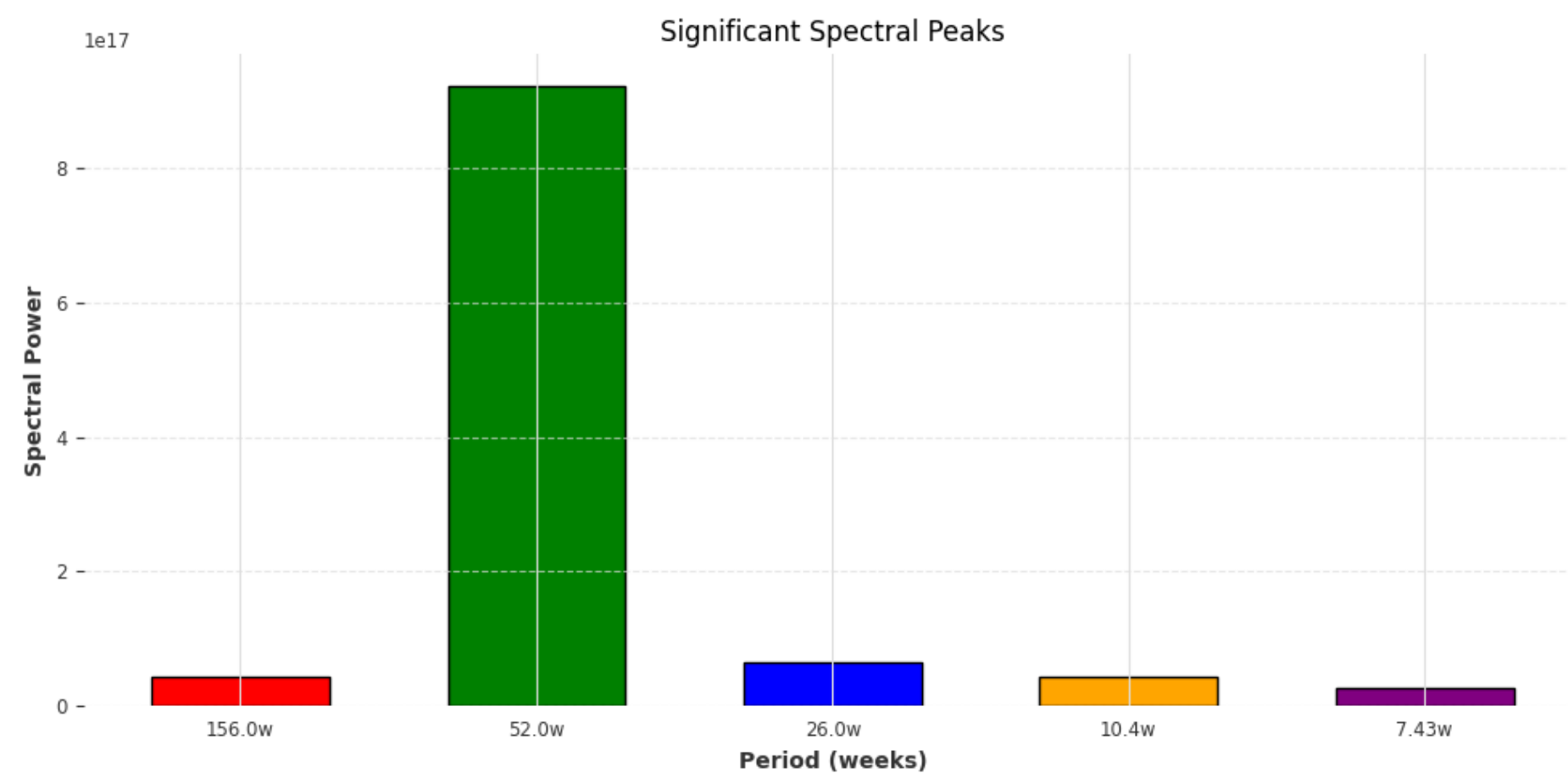
Frequency: 0.00641, Power: 4.29e+16, Period ≈ 156.00 weeks
Frequency: 0.01923, Power: 9.24e+17, Period ≈ 52.00 weeks
Frequency: 0.03846, Power: 6.33e+16, Period ≈ 26.00 weeks
Frequency: 0.09615, Power: 4.15e+16, Period ≈ 10.40 weeks
Frequency: 0.13462, Power: 2.65e+16, Period ≈ 7.43 weeks



Reconstructed series from most significant frequencies

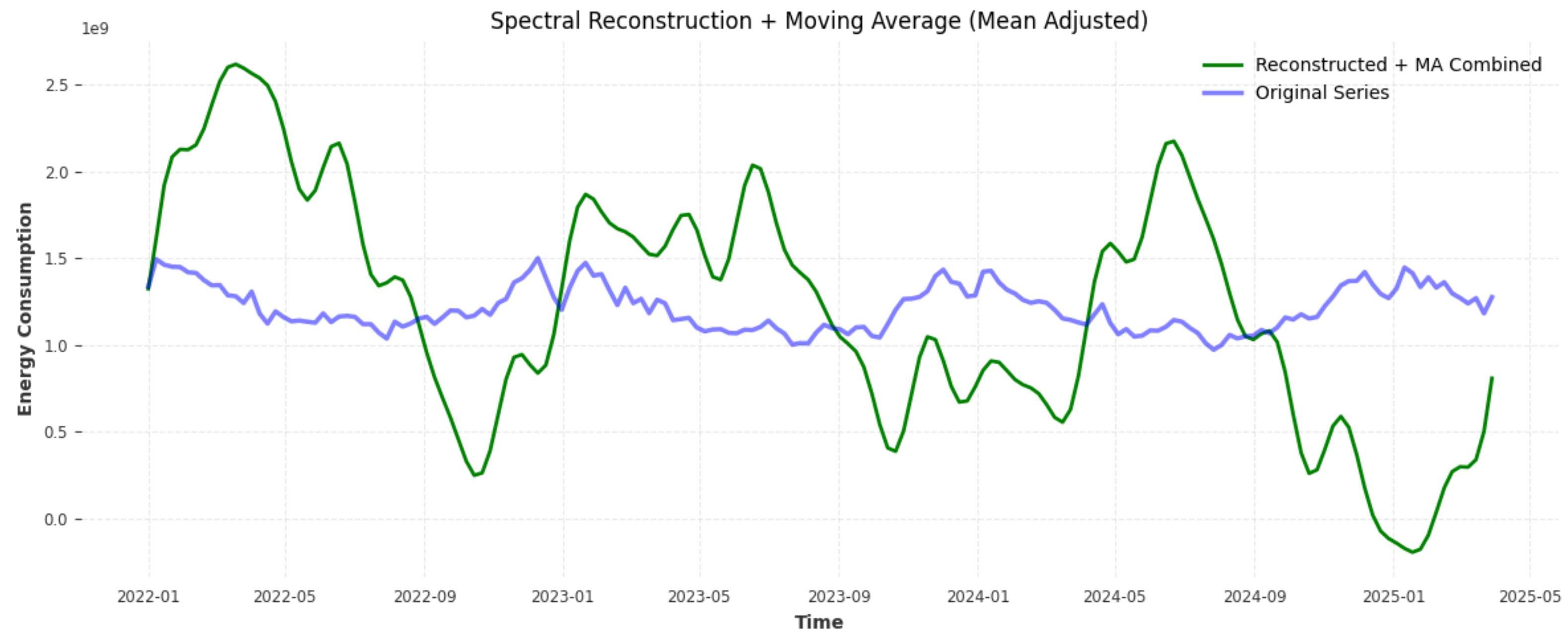
Result

- The spectral decomposition shows that the highest frequency is of 52 weeks, this clearly demonstrates a yearly seasonality.
- Explain why differenced vs original frequencies similar, rigorous argument



Reconstructed series from most significant frequencies

Result



Testing for white noise

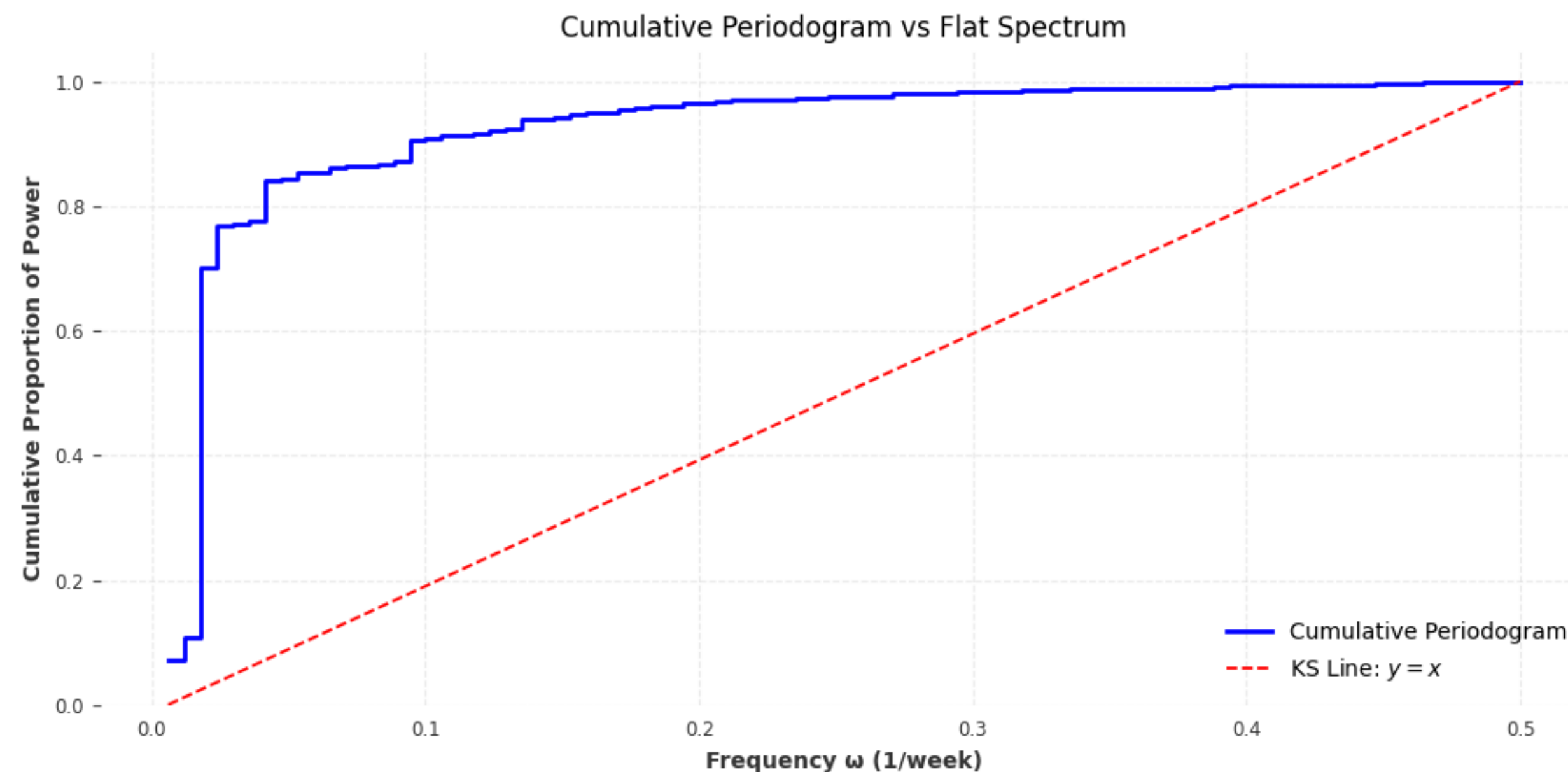
Cumulative Periodogram compared to Gaussian White noise spectrum

(c) The cumulative periodogram

$$C_r = \frac{\sum_{j=1}^r I(\omega_j)}{\sum_{l=1}^m I(\omega_l)}, \quad r = 1, \dots, m$$

is a plot of $C_1, \dots, C_m$ against the frequencies ω_j for $j = 1, \dots, m$. [4]

According to Davidson, Gaussian and non-Gaussian white noise has a flat spectrum [4]

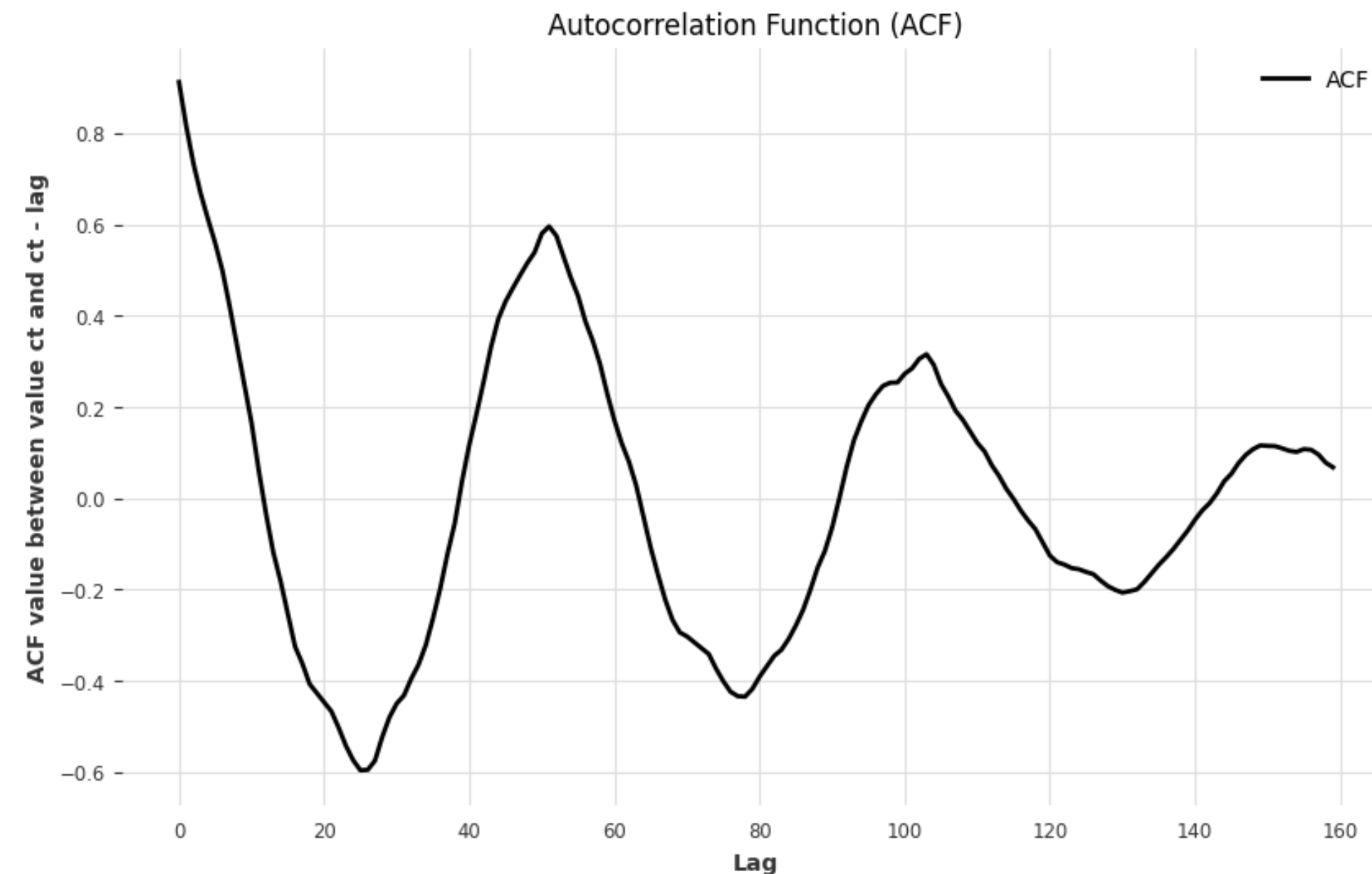


will be redone after removing periodicity

the frequency is very high near 1/52 weeks, suggesting 52 seasonality again, and also proving that our data is again not white noise

Determining Seasonal order (P)

Original ACF

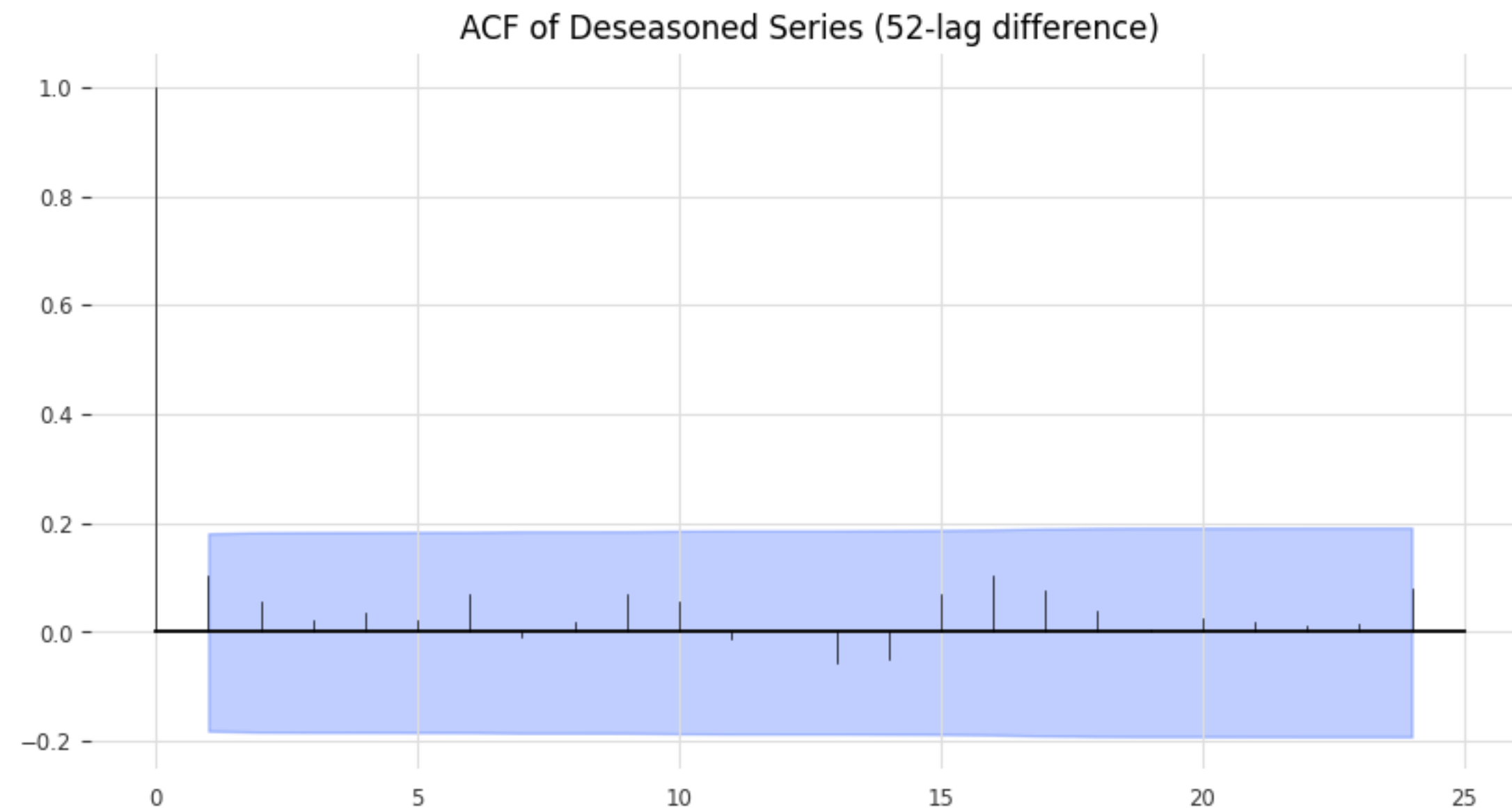


At lag 52, ACF = 0.6, the rest of the annual points are much lower and decay to 0
To determine which seasonal order to use (how many seasonal yearly lags to use
I've set a threshold of Autocorrelation of minimum 0.5

====> $P = 1$

Testing for stationnarity and D order

ACF of deseasoned data



```
from darts.utils.statistics import stationarity_tests

is_stationary = stationarity_tests(
    initial_series.diff(52),
    p_value_threshold_kpss=0.05
)

# Print interpretation
print("Series is stationary" if is_stationary else "Series is not stationary")
```

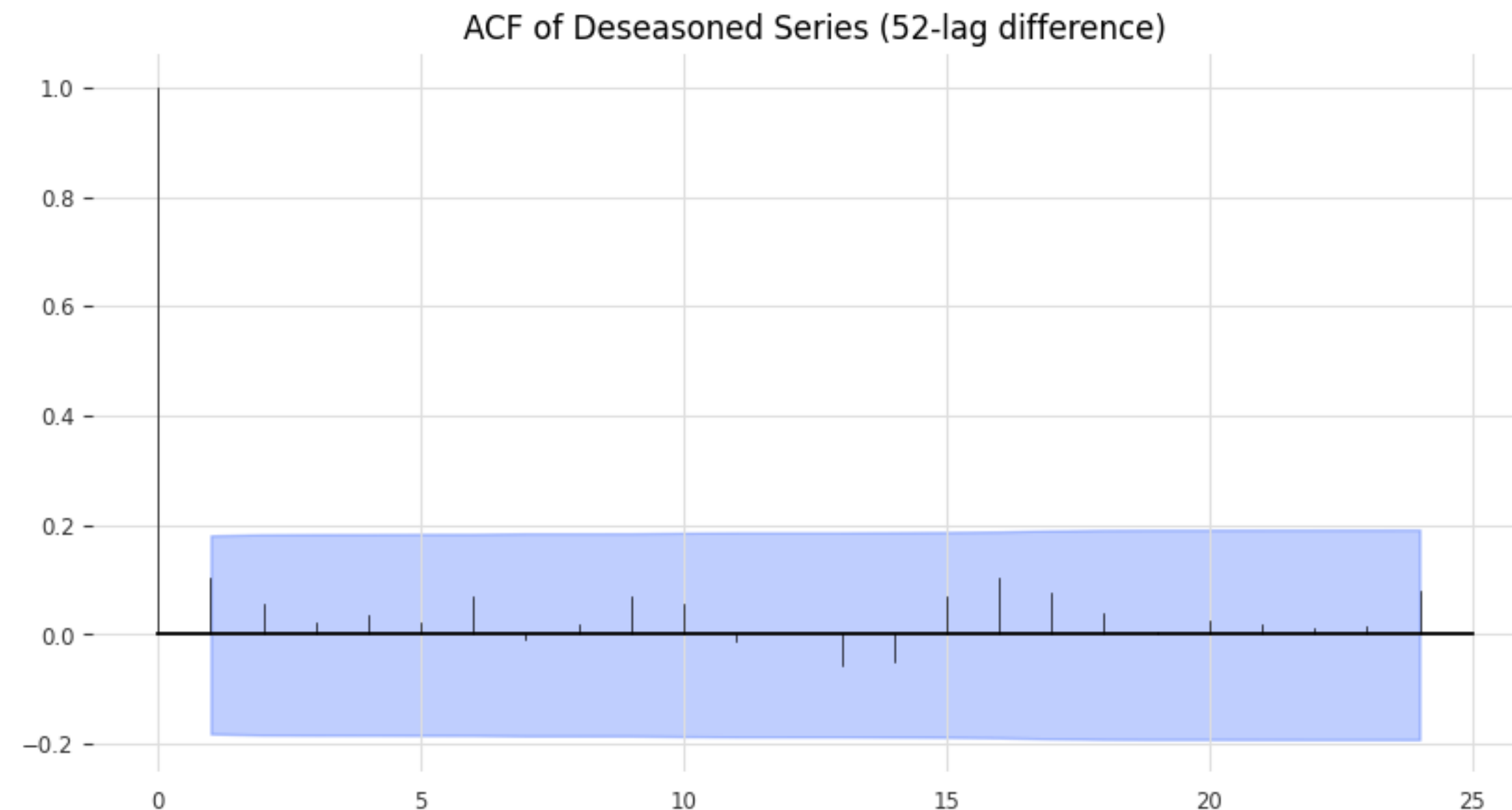
✓ 0.1s
Series is stationary

The ACF values of the deseasoned ts show no significant cutout after a certain lag, this does not suggest a ARMA model

D = 1

Testing for MA order (Q)

ACF of deseasoned data



The ACF values of the deseasoned ts show no significant cutout after a certain lag, this does not suggest a ARMA model

$$Q = 0$$

Interpretation

Result

SARIMA (1,0,0 [1,1,0,52])

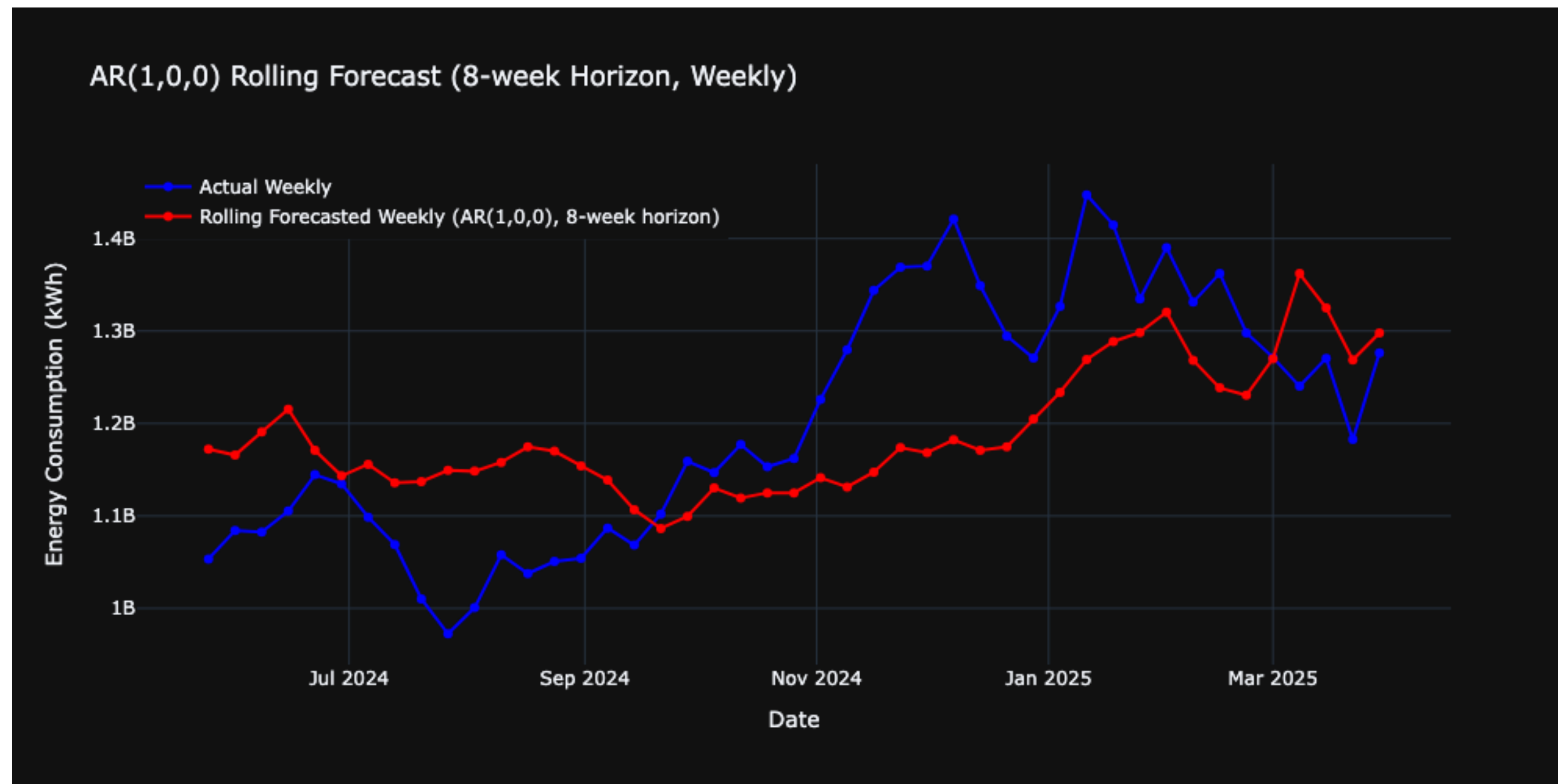
- From the Cumulative periodogram, we concluded our TS is not white noise
- So from the stationarity test, we concluded that d is likely 0, from deseasoned ACF, we concluded that $q = 0$, and from PACF we concluded that p is likely 1 (plot cumu. period for des ACF)
- From Spectral Analysis, we concluded that the seasonality is clearly 52 weeks, these important results will help us create our forecasting models.

IV. Models

AR

Weekly sum

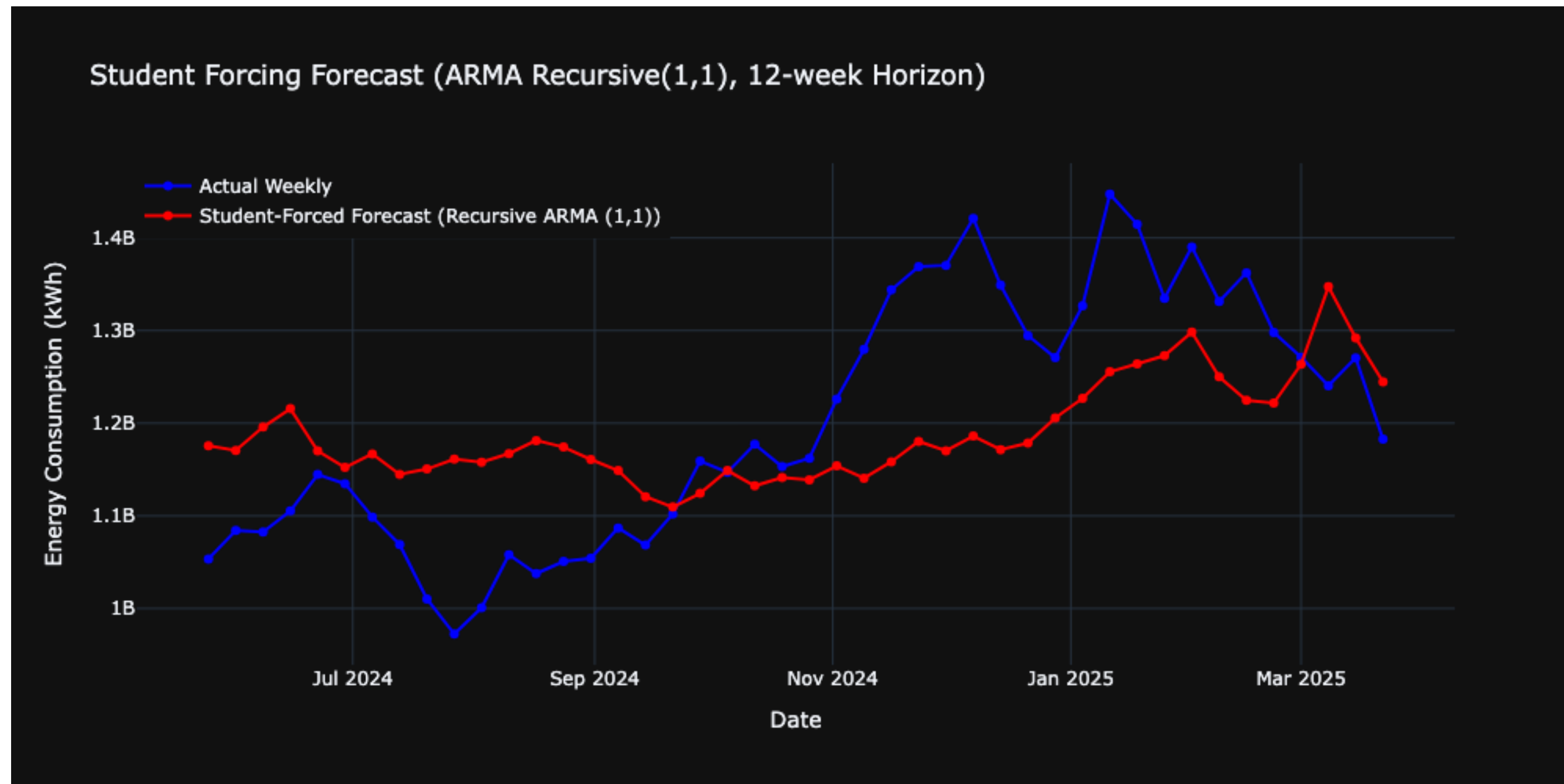
$$X_t = \sum_{i=1}^p \varphi_i X_{t-i} + \varepsilon_t$$



ARMA

Weekly sum

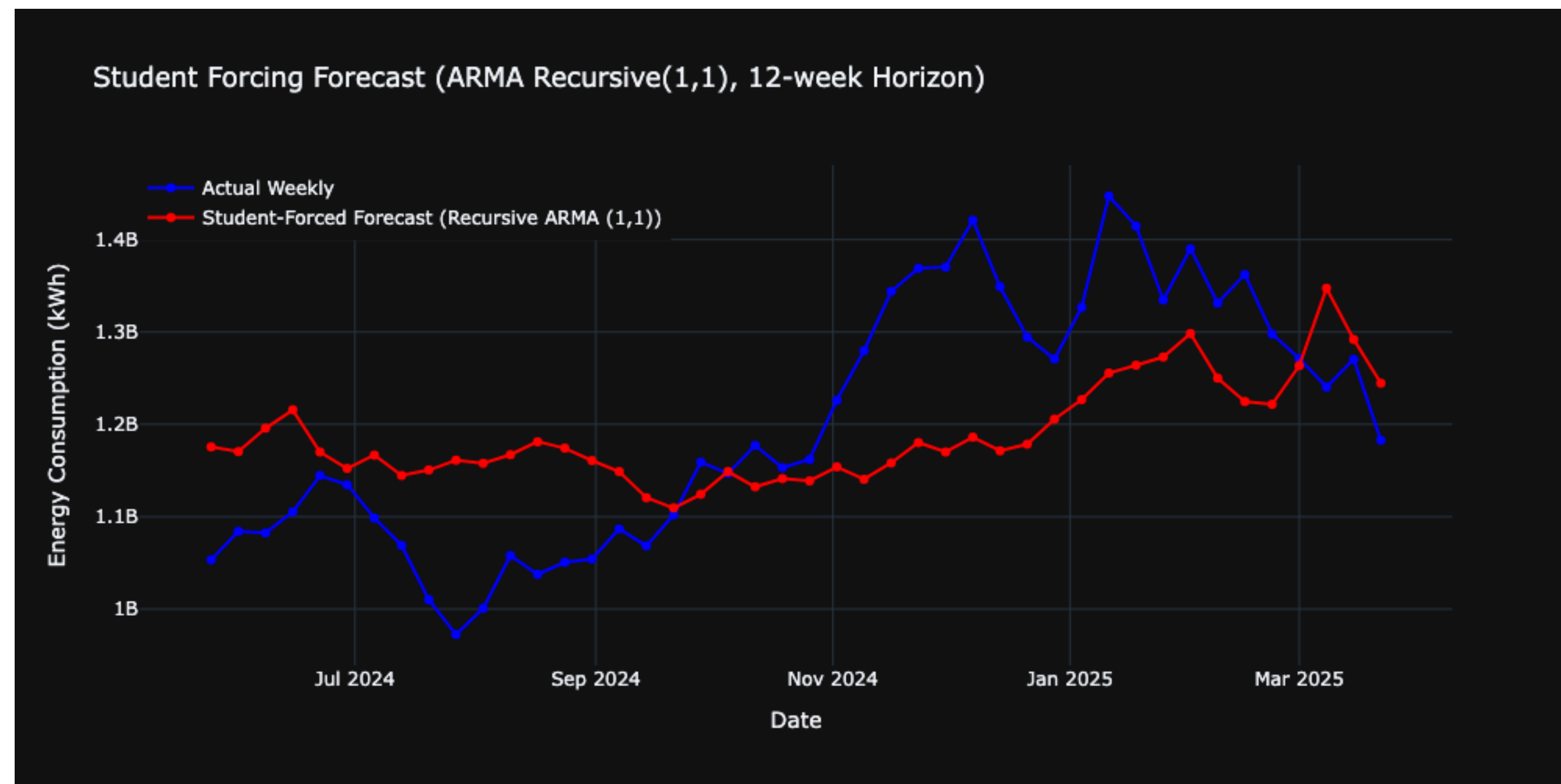
$$X_t = \varepsilon_t + \sum_{i=1}^p \varphi_i X_{t-i} + \sum_{i=1}^q \theta_i \varepsilon_{t-i}.$$



ARIMA

Weekly sum

$$\left(1 - \sum_{i=1}^{p'} \alpha_i L^i\right) X_t = \left(1 + \sum_{i=1}^q \theta_i L^i\right) \varepsilon_t$$



SARIMA

Definition

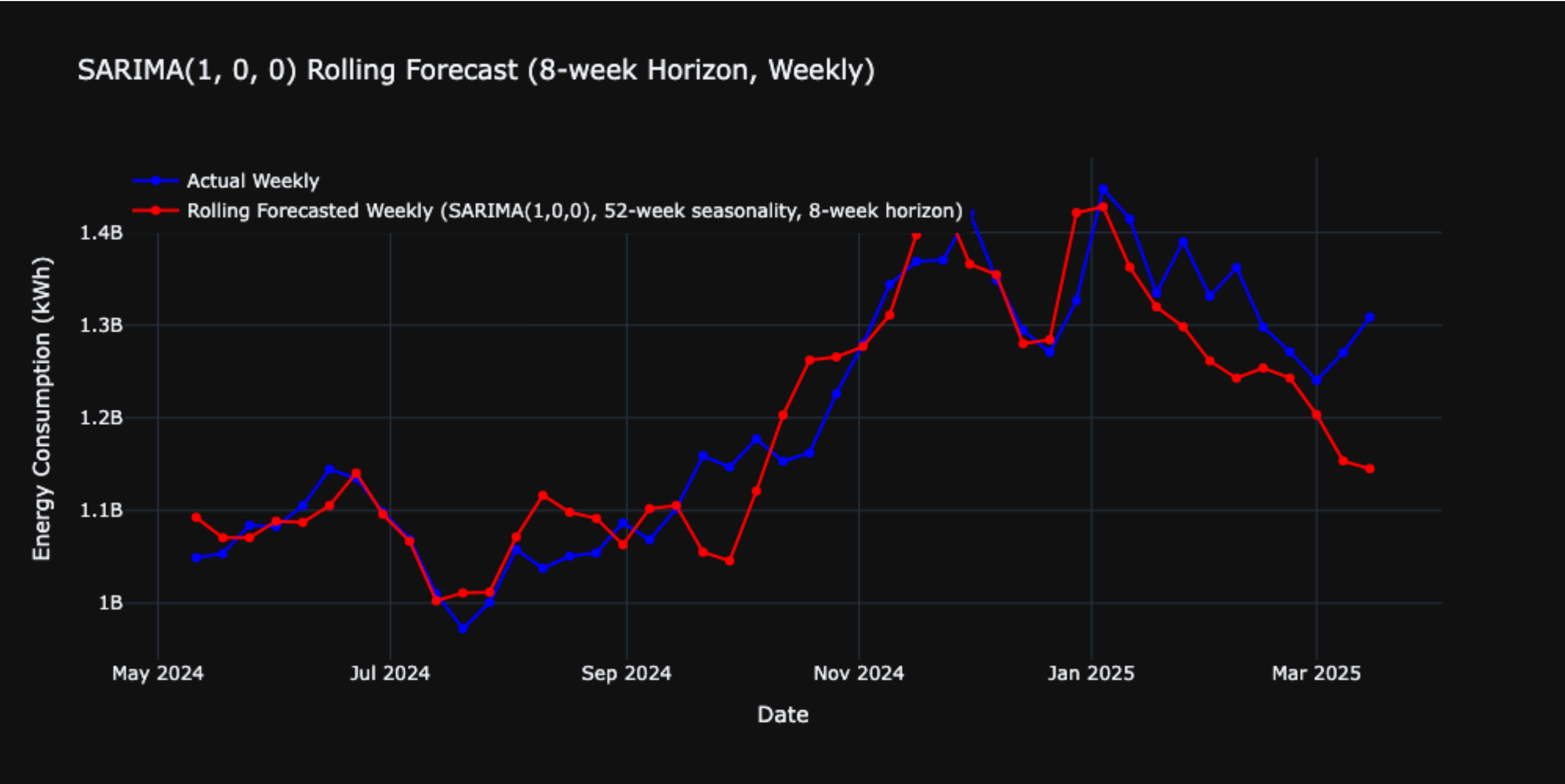
The multiplicative seasonal autoregressive moving average model $\text{SARIMA}(p, d, q) \times (P, D, Q)_s$ is

$$\Phi_P(B^s)\phi(B)(I - B)^d(I - B^s)^DY_t = \alpha + \Theta_Q(B^s)\theta(B)\varepsilon_t,$$

where $\{\varepsilon_t\}$ is Gaussian white noise. The ordinary autoregressive and moving average components are represented by the operators $\phi(B)$ and $\theta(B)$, respectively; the seasonal autoregressive and moving average components by $\Phi_P(B^s)$ and $\Theta_Q(B^s)$, of orders P and Q ; and the ordinary and seasonal difference components by $(I - B)^d$ and $(I - B^s)^D$ of orders d and D .

SARIMA

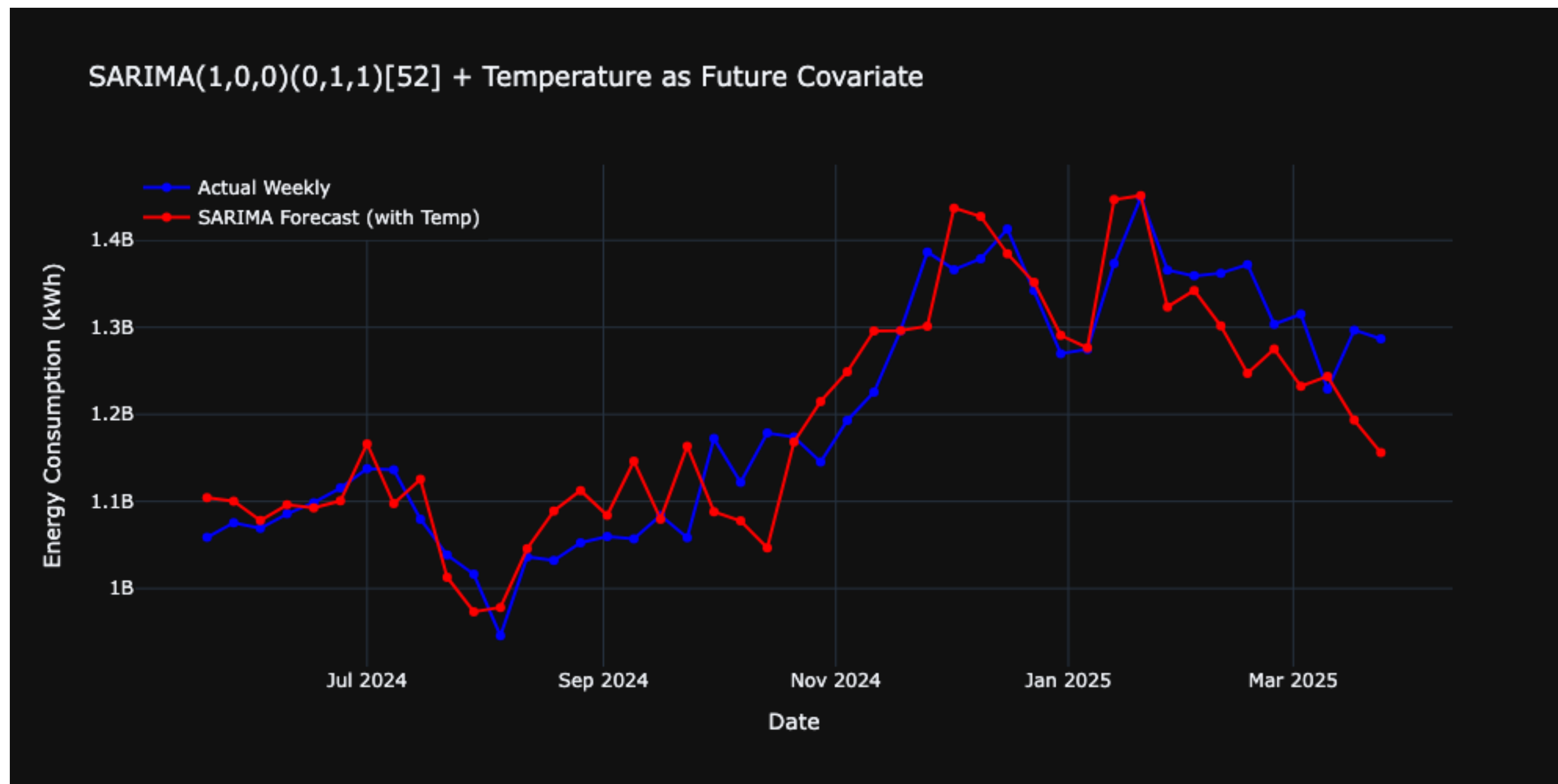
Weekly sum



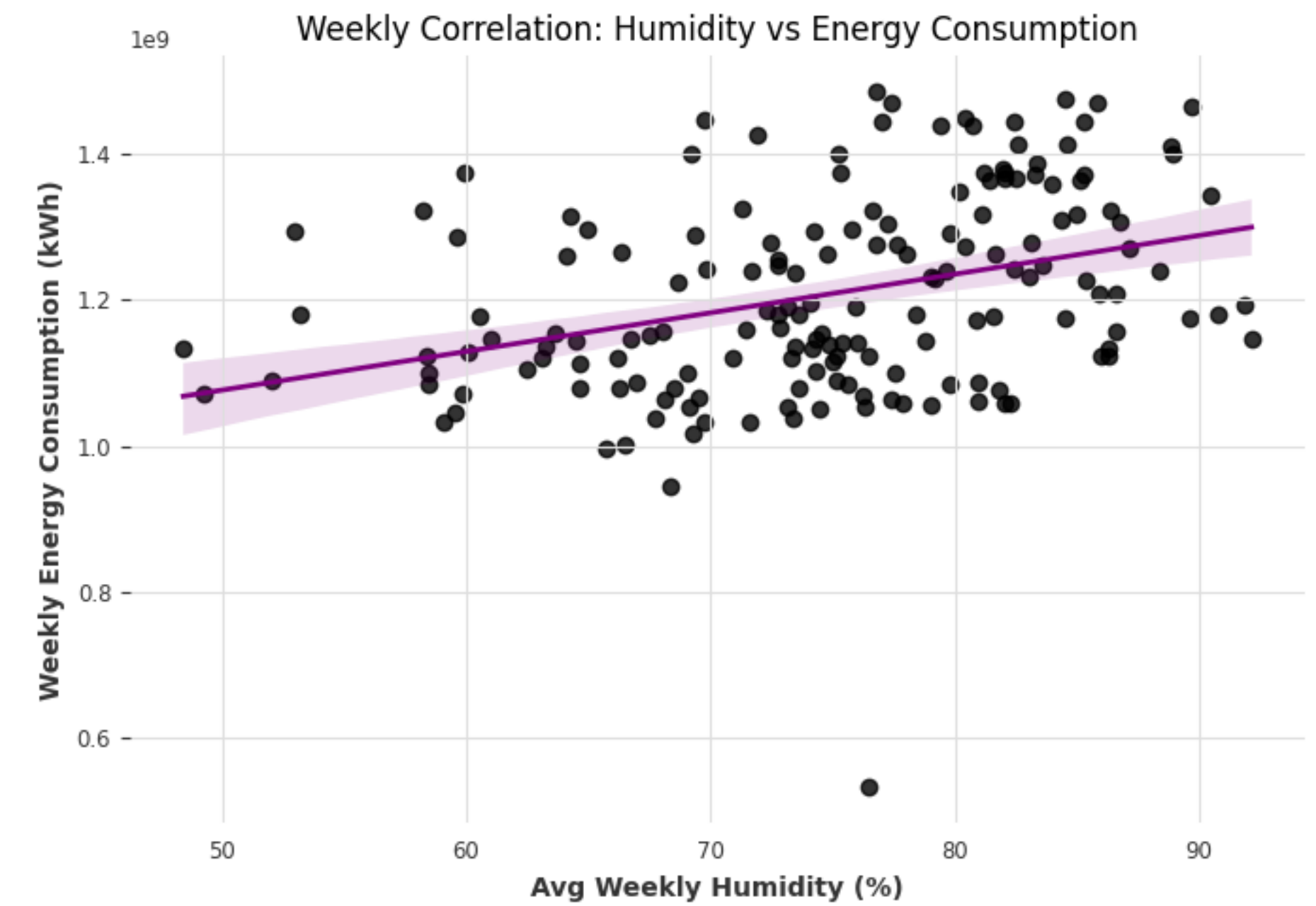
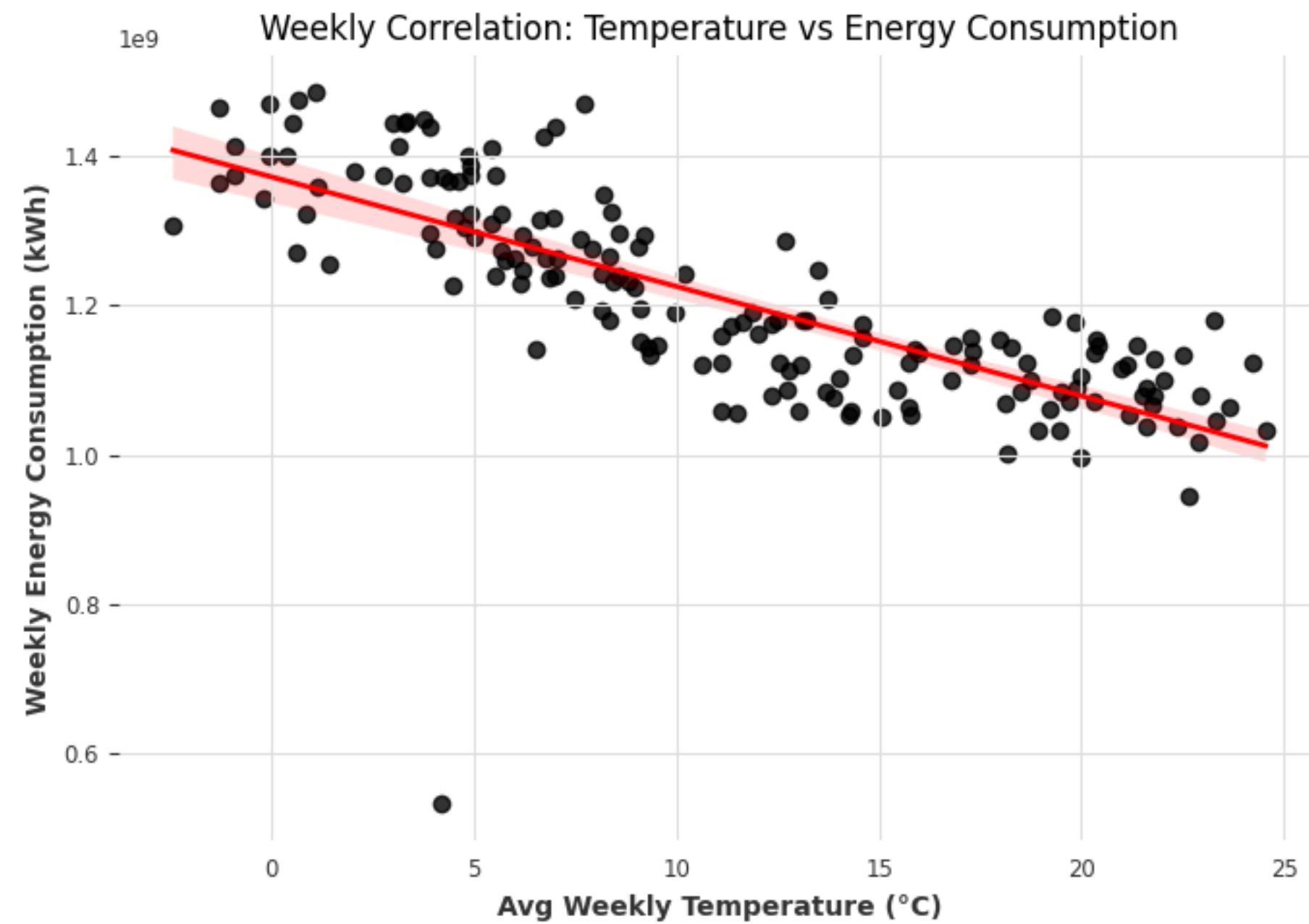
SARIMAX

Weekly sum

$$\Theta(L)^p \theta(L^s)^P \Delta^d \Delta_s^D y_t = \Phi(L)^q \phi(L^s)^Q \Delta^d \Delta_s^D \epsilon_t + \sum_{i=1}^n \beta_i x_t^i$$

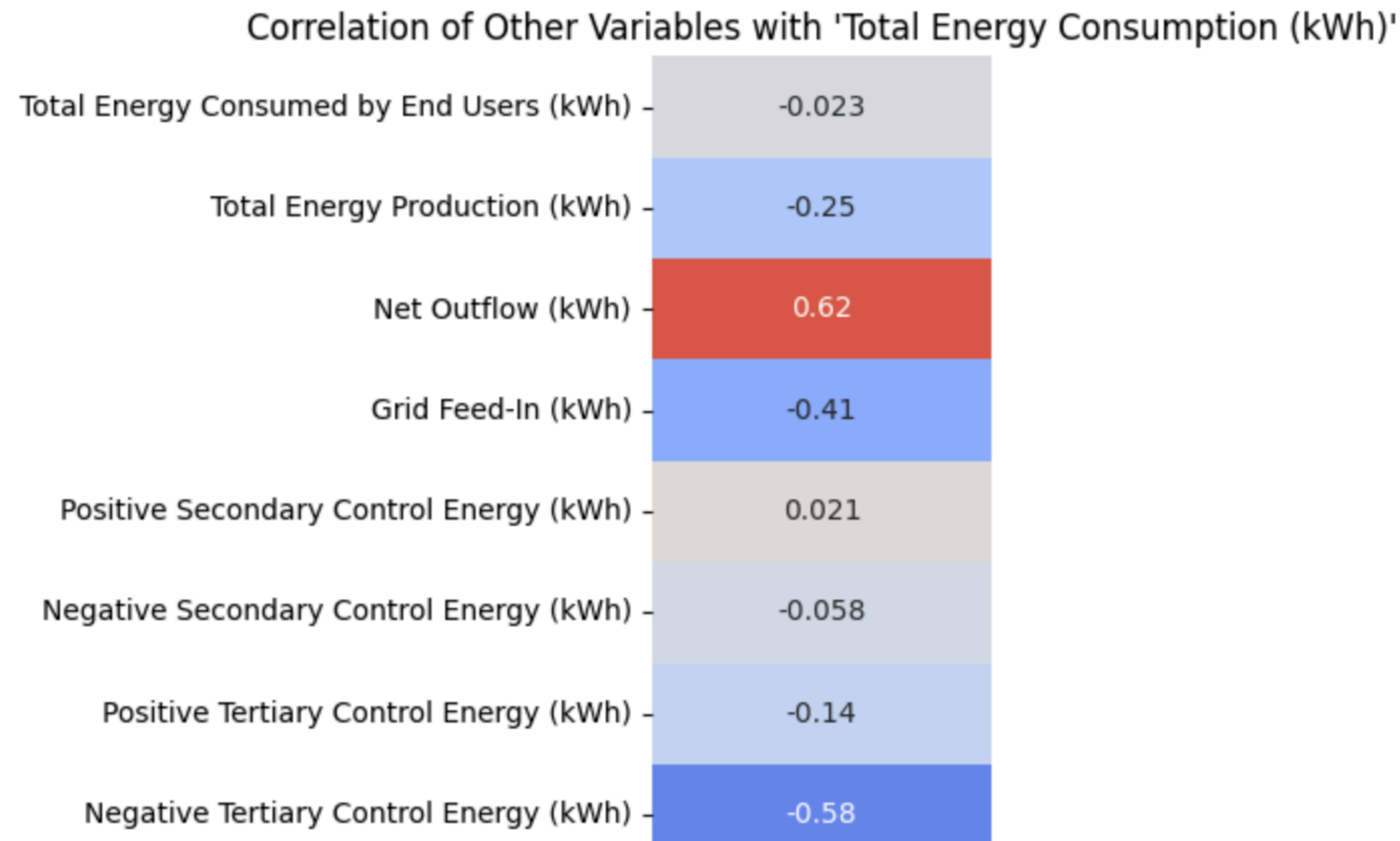


Correlations

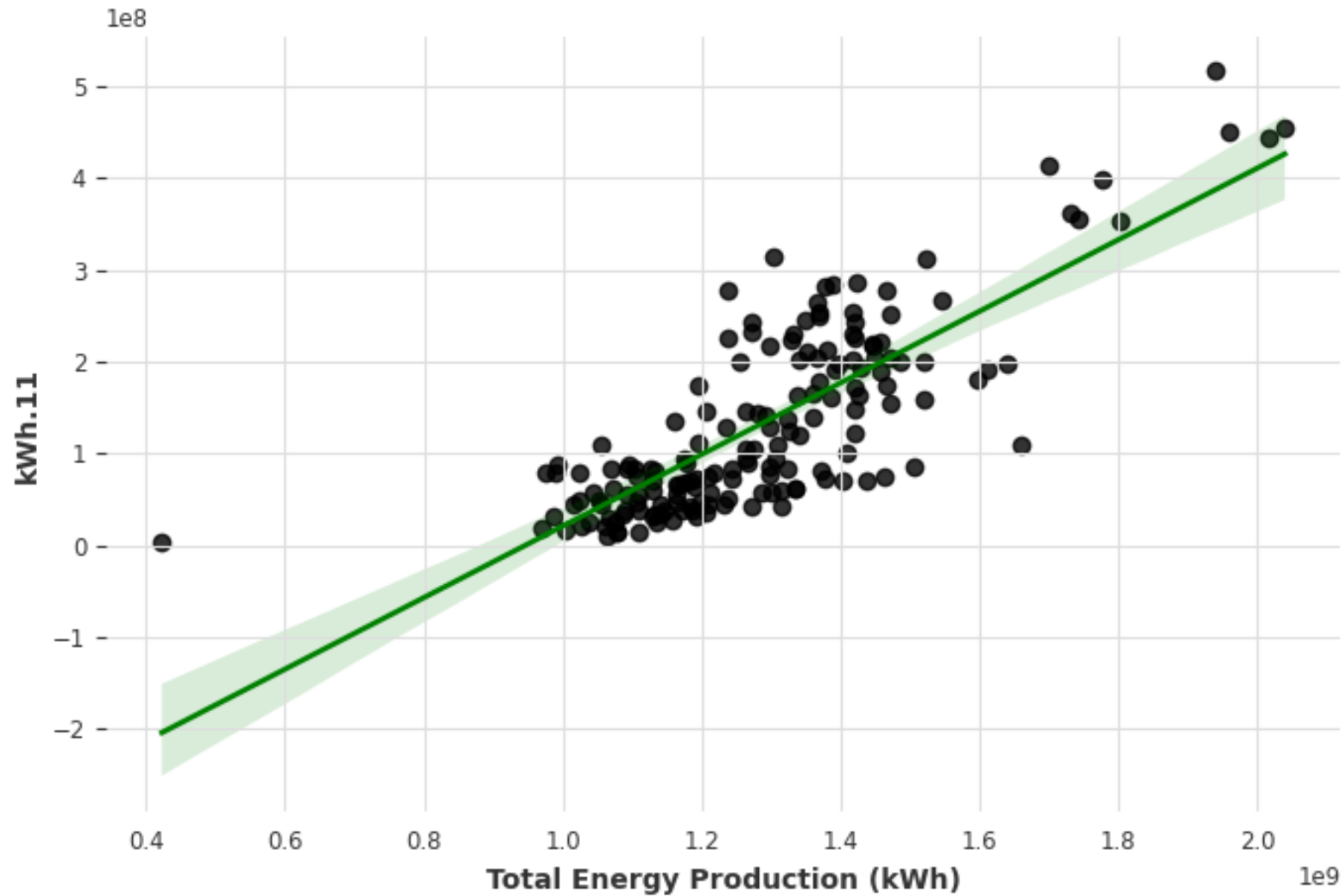


```
✓ Pearson correlation:  
energy  
temperature -0.750454  
humidity    0.350662
```


Correlations with other variables in the same data file



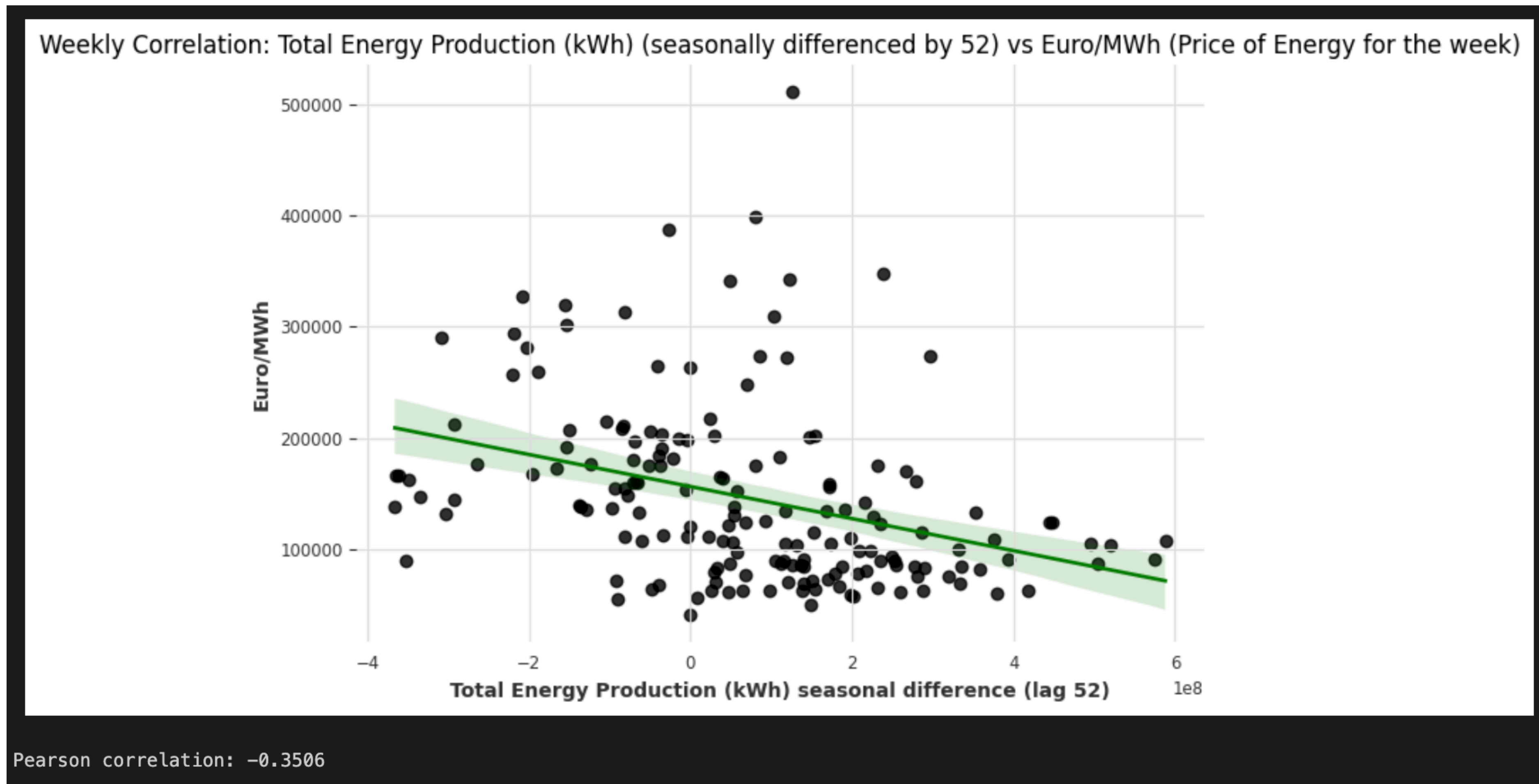
Weekly Correlation: Total Energy Production (kWh) vs kWh.11 Cross Border Exchange CH->DE



Pearson correlation: 0.8053

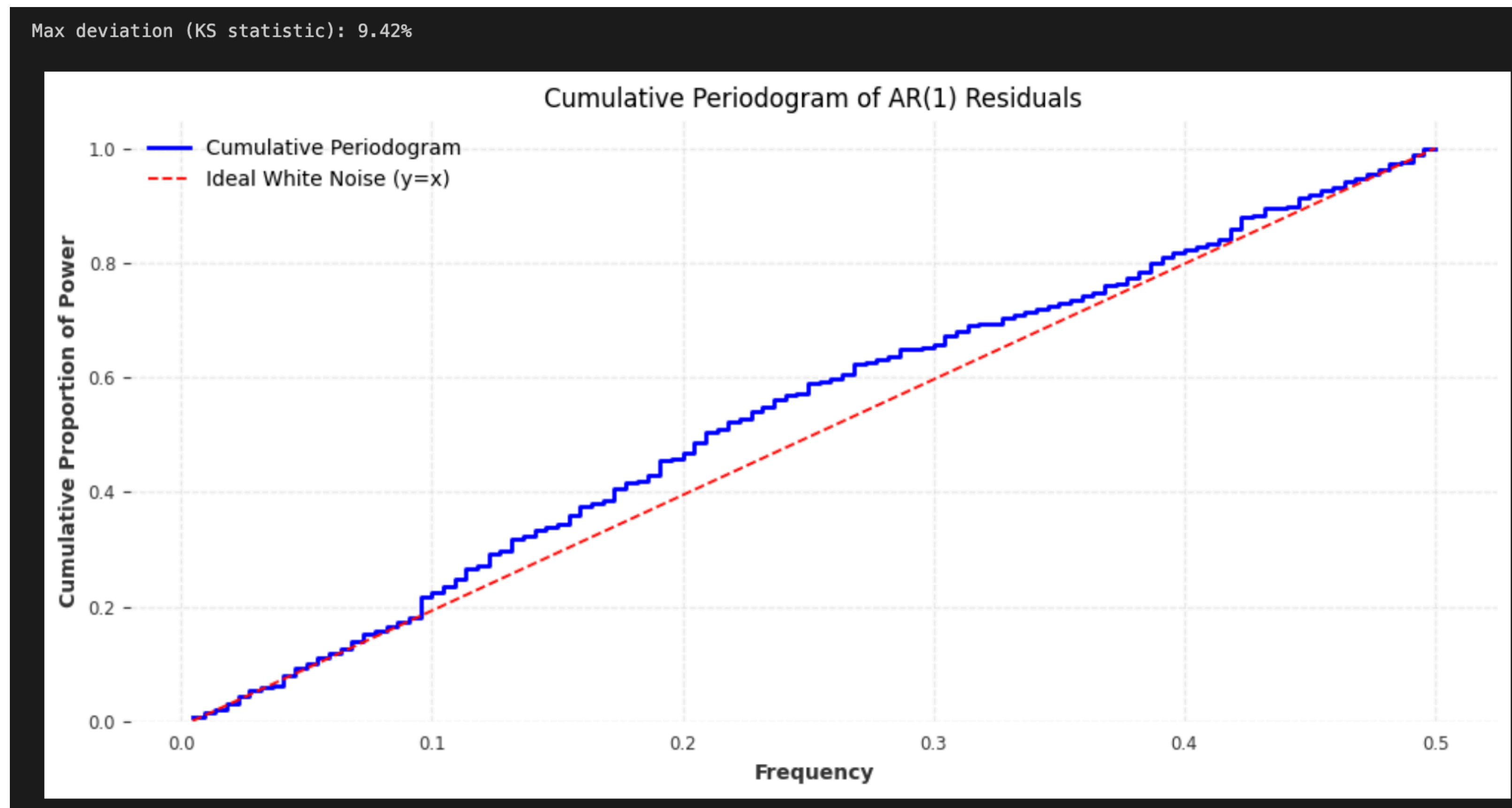
Exogenous Variables

if remove seasonality is there correlation with energy prices?



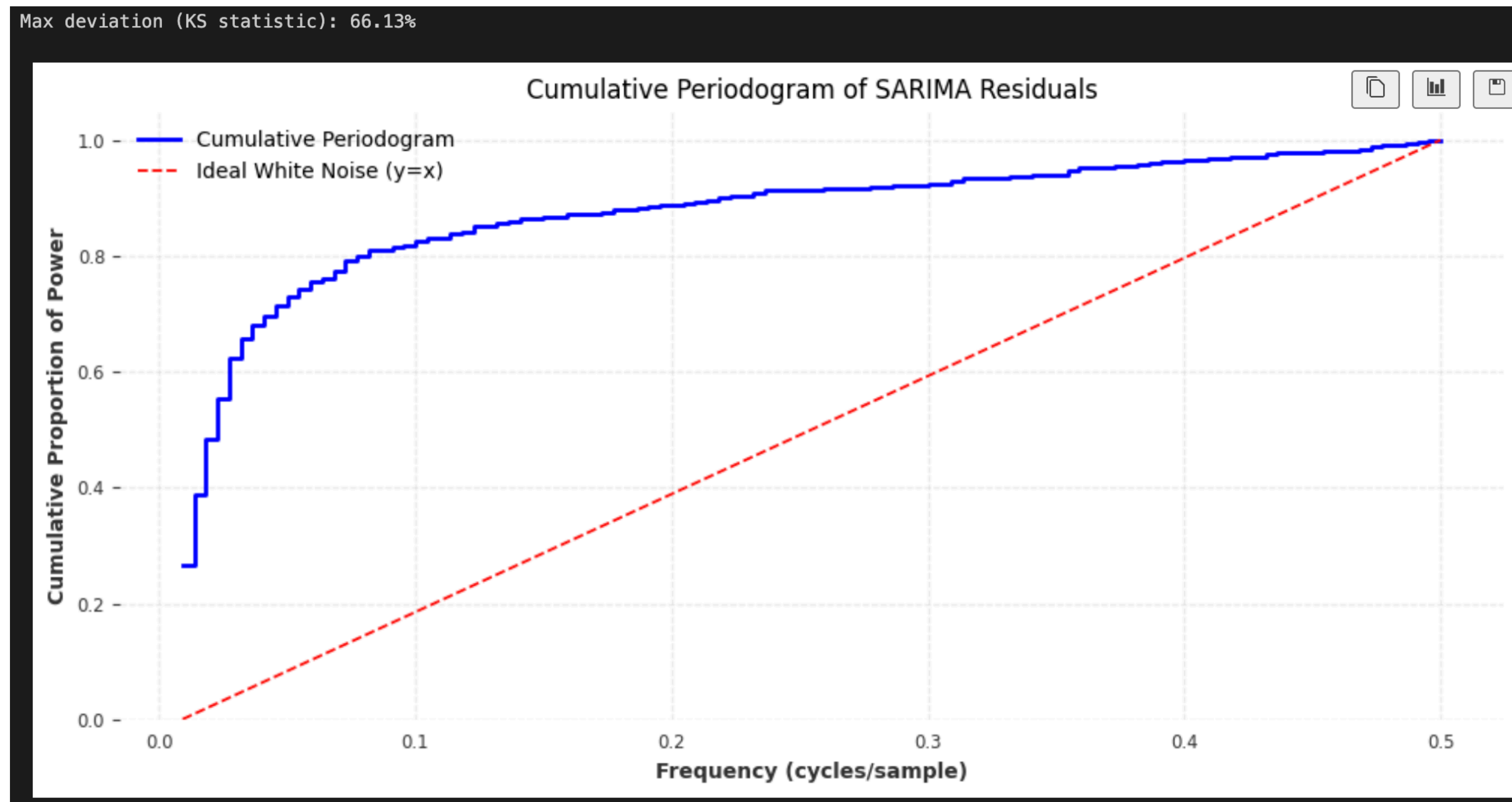
Testing model residuals for white noise

AR(1)



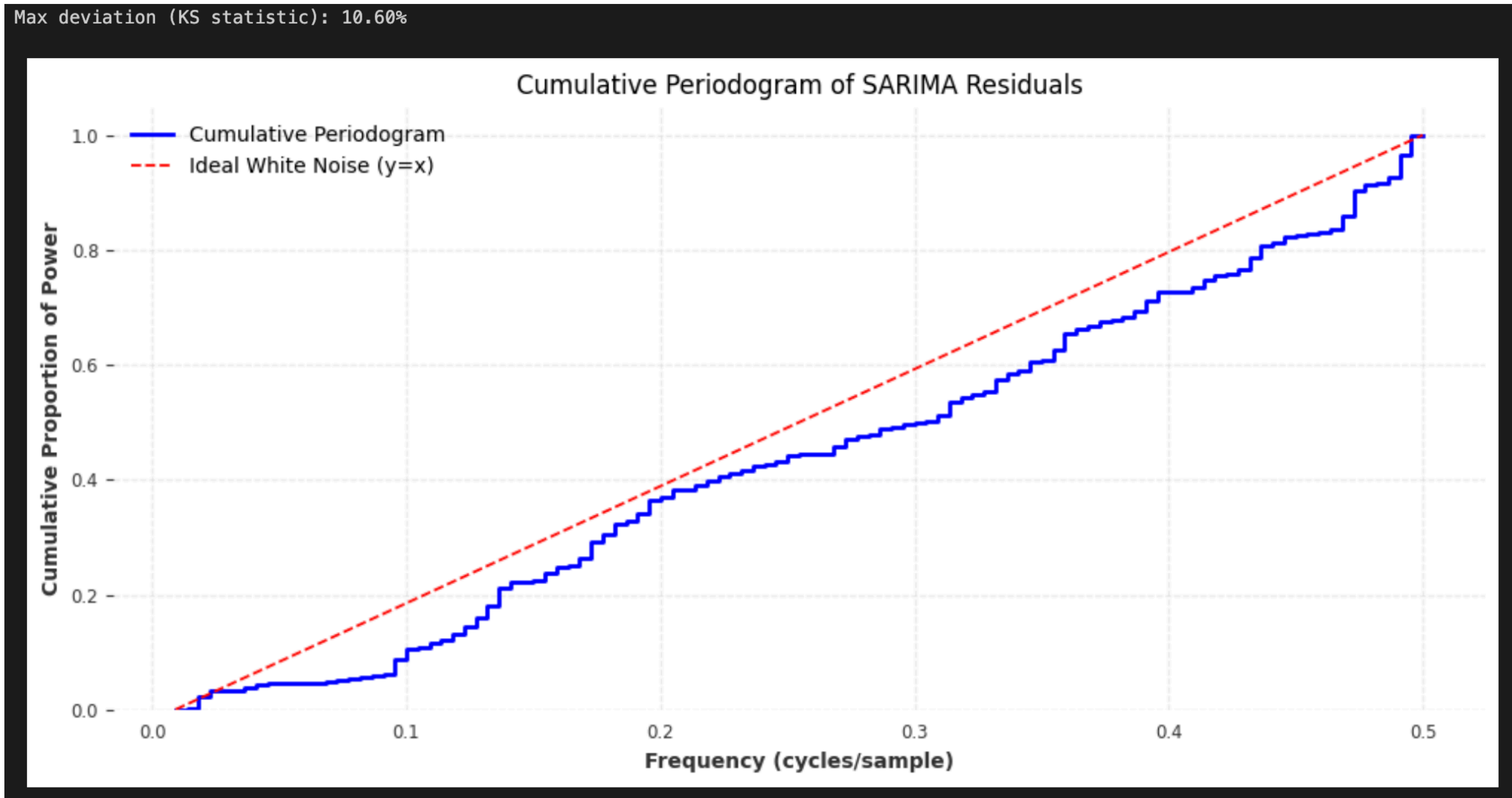
Testing model residuals for white noise

SARIMA(1,0,0, (1,1,0,52))



Testing model residuals for white noise

SARIMA(1,0,0, (1,0,0,52))



V. Evaluation

MAPE

Evaluation Metric

7.1.1 Definition

The mean absolute percentage error (MAPE) is a metric used to evaluate the accuracy of a forecasting model. It calculates the average of the absolute percentage errors between the predicted values and the actual values. Lower MAPE values indicate more accurate forecasts.

The formula is:

$$\text{MAPE} = \frac{100\%}{n} \sum_{t=1}^n \left| \frac{y_t - \hat{y}_t}{y_t} \right|$$

where y_t is the true value and $\hat{y}_t$ is the predicted value at time t . [15]

MAPE

Evaluation Metric

Model Comparison – MAPE Scores

Model	MAPE (%)
AR(1)	8.01
ARMA(1,1)	8.12
ARIMA(1,1,1)	8.16
SARIMA(1,0,0 [0,1,1,52])	3.53
SARIMAX(1,0,0 [0,1,1,52]) with Temp	3.86

We concluded that the SARIMA (1,0,0,[52]) is best

MAPE

Evaluation Metric

The baseline AR(1) model yielded a modest prediction accuracy, providing a simple yet interpretable benchmark. However, our selected SARIMA(1, 0, 0)(1, 1, 0, 52) model outperformed all baseline and intermediate models, achieving a MAPE that was approximately 237% better than the AR(1) benchmark.

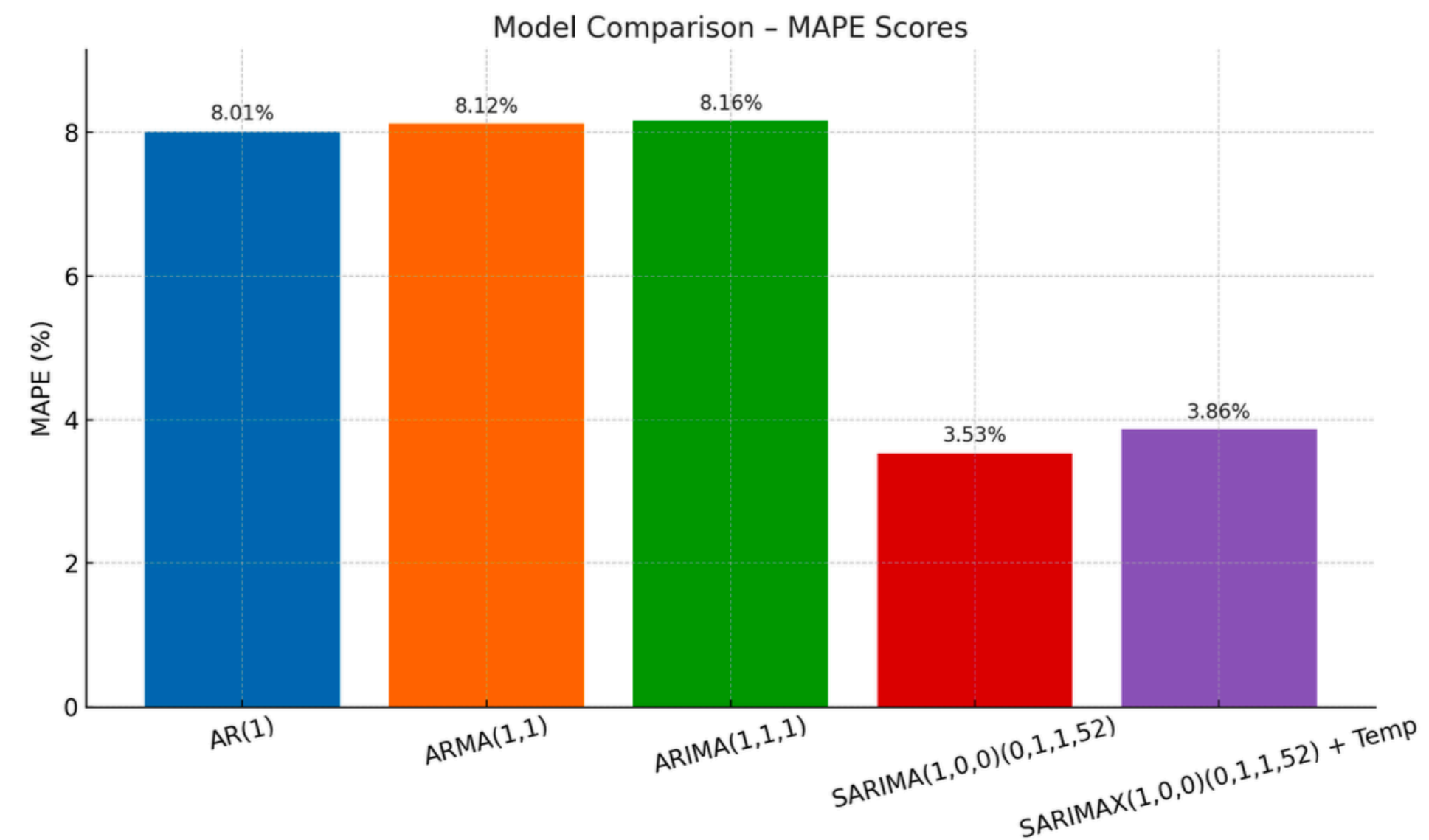


Figure 7: MAPE scores graph

We concluded that the SARIMA (1,0,0,[52]) is best

IV. Challenges and discussion

Challenges

- I. Understanding Time Series Analysis and what each formula represents
- II. Finding conflicting explanations online
- III. Determining AR weights using Gradient descent instead of Yule-Walker Equation
- IV. Understanding academic analysis: doing research on other papers, reports using LaTeX, etc.
- V. Backing up my approaches and results using scientific methods

Thanks for coming !